

MACHINE LEARNING, TOLD SIMPLY

Book 3 • Contact Sessions 3 & 4

# Drawing the Best Line

*Linear models for regression — the cost function, the closed-form solution, gradient descent and its variants, evaluation, polynomial and basis-function models, bias–variance, and regularisation.*

---

Machine Learning • AIML ZG565 / DSE ZG565 • BITS Pilani WILP

Companion to Contact Sessions 3 & 4 (Module 3: Linear Models for Regression)

Companion notes prepared by Prof. Saurabh

## What is inside

Meera finally fits her first model. This book takes every formula on the Session 3–4 slides and builds it twice: first with plain numbers, one step at a time, then in the matrix form the computer actually runs. Every slide example is worked in full — the tips data, the salary data, the tiny house data, the BMI/blood-pressure problem, with and without regularisation.

|           |                                                                    |                                                                                                                                     |
|-----------|--------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| <b>1</b>  | <b>A Line With Two Knobs</b>                                       | The hypothesis, its parameters, and the question every regression asks: how do we choose the knobs?                                 |
| <b>2</b>  | <b>What Does "Best" Mean? The Cost Function</b>                    | Residuals, squared errors, the cost $J(\theta)$ — and why it is a bowl.                                                             |
| <b>3</b>  | <b>Route 1: Solve It — The Closed-Form Solution</b>                | Set the derivatives to zero and solve. First for one input (two formulas), then for any number of inputs (the normal equation).     |
| <b>4</b>  | <b>Route 2: Walk Downhill — Gradient Descent</b>                   | The update rule, where it comes from, why it always goes downhill, and a full numerical run.                                        |
| <b>5</b>  | <b>Making Descent Behave: Learning Rate and Feature Scaling</b>    | Problem Type 2 — reading a convergence plot and diagnosing the hyper-parameters.                                                    |
| <b>6</b>  | <b>The Exam Problem: Two Predictors, One Iteration</b>             | Problem Type 3 — the BMI / blood-pressure regression worked to the last decimal, plus the batch / stochastic / mini-batch variants. |
| <b>7</b>  | <b>How Good Is the Line? Evaluating Regression</b>                 | MSE, MAE, RMSE and $R^2$ — what each one measures and how to compute them by hand.                                                  |
| <b>8</b>  | <b>Bending the Line: Basis Functions and Polynomial Regression</b> | How a "linear" model fits a curve — linear in the weights, not in the inputs.                                                       |
| <b>9</b>  | <b>Bias, Variance, and the Art of Not Overfitting</b>              | Problem Type 4 — reading learning curves and model-fit pictures, and the two easy cures for overfitting.                            |
| <b>10</b> | <b>Regularisation: Ridge, Lasso, Elastic Net</b>                   | Penalise big weights, derive the changed update rules, and finish the BMI problem with $\lambda = 5$ .                              |
| <b>11</b> | <b>Formula Sheet, Self-Check, and What Comes Next</b>              | Every formula from Sessions 3–4 on one page, twelve questions with answers, and the bridge to classification.                       |

## How to read this book

Every chapter follows the same rhythm: a short **story** to set the scene, the **big idea** in one line, an everyday **analogy**, then the **maths step by step** — always the plain version first, and the matrix version only after the plain one is clear. Boxes to watch for:

**STORY**

Meera at Sahi Deal — the running example.

**BIG IDEA**

The one sentence to remember.

**THINK OF IT LIKE**

A real-life picture of the concept.

**STOP AND THINK**

A question to try before reading on.

**OUR ANSWER**

The answer, right after the question.

**CAREFUL**

A common mistake and how to avoid it.

**MATHS, STEP BY STEP**

Every line of algebra, nothing skipped.

**WORKED EXAMPLE**

Real numbers, worked in full.

**EXAM TIP**

How this topic is asked, and how to score.

**IN ONE BREATH**

The chapter in a few lines.

## CHAPTER 1

# A Line With Two Knobs

*The hypothesis, its parameters, and the question every regression asks: how do we choose the knobs?*

**STORY**

Meera's scatter plot is on the wall: 40,000 dots, kilometres on the horizontal axis, price on the vertical. The cloud slopes downward. Arjun hands her a ruler. "Draw the line that best goes through it." Meera lays the ruler down, tilts it, shifts it up... "How do I know when it is *best*?" "Good. That is the entire subject."

## The hypothesis

In regression we are given pairs  $(x^{(1)}, y^{(1)}), \dots, (x^{(n)}, y^{(n)})$  and want a function that predicts a real-valued  $y$  from  $x$ . The slides use the house-price example:

| Size in feet <sup>2</sup> ( $x$ ) | Price in 1000s ( $y$ \$) |
|-----------------------------------|--------------------------|
| 2104                              | 460                      |
| 1416                              | 232                      |
| 1534                              | 315                      |
| 852                               | 178                      |
| ...                               | ...                      |

The simplest useful guess is a straight line. We call it the **hypothesis** and write it as

$$h_{\theta}(x) = \theta_0 + \theta_1 x.$$

$h_{\theta}(\cdot)$  The model — a rule that turns an input into a predicted output.

$\theta_0$  The **intercept** (bias): the predicted  $y$  when  $x = 0$ . Where the line crosses the vertical axis.

$\theta_1$  The **slope** (weight): how much  $y$  changes when  $x$  goes up by one unit.

**BIG IDEA**

$\theta_0$  and  $\theta_1$  are the **parameters** — the knobs. "Learning" a linear regression means nothing more than choosing numbers for these two knobs. Everything in this book is about *how* to choose them well.

**THINK OF IT LIKE**

A taxi fare. There is a flag-down charge ( $\theta_0$ , paid even for zero distance) and a per-kilometre rate ( $\theta_1$ ). Fare =  $\theta_0 + \theta_1 \times \text{distance}$ . The taxi company "learnt" those two numbers from years of costs. Given any distance, the meter predicts the fare instantly — that is  $h_{\theta}(x)$ .

## Simple versus multiple regression

With one input (education → income) it is **simple linear regression**. With several inputs (education, soft skills, experience, age → income) it is **multiple linear regression**: one weight per input, plus the intercept:

$$h_{\theta}(\mathbf{x}) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_d x_d.$$

Same idea, more knobs. Meera's price model will use  $d$  inputs — age, engine size, kilometres — and  $d + 1$  parameters.

### A TIDY TRICK — THE "1" FEATURE

Writing  $\theta_0$  separately is untidy. Invent a feature  $x_0$  that is always 1. Then

$$h_{\theta}(\mathbf{x}) = \theta_0 x_0 + \theta_1 x_1 + \cdots + \theta_d x_d = \sum_{j=0}^d \theta_j x_j = \boldsymbol{\theta}^T \mathbf{x},$$

with  $\boldsymbol{\theta} = (\theta_0, \theta_1, \dots, \theta_d)^T$  and  $\mathbf{x} = (1, x_1, \dots, x_d)^T$ . A prediction is a dot product. Keep this in mind — the matrix forms in Chapters 3 and 4 depend on it.

## Parameters versus hyper-parameters

The **parameters**  $\theta_j$  are set by the learning algorithm from data. Things you set by hand *before* learning — the learning rate  $\alpha$ , the polynomial degree  $M$ , the regularisation strength  $\lambda$  — are **hyper-parameters**. The exam loves this distinction: " $\alpha$  is a hyper-parameter;  $\theta_1$  is a parameter."

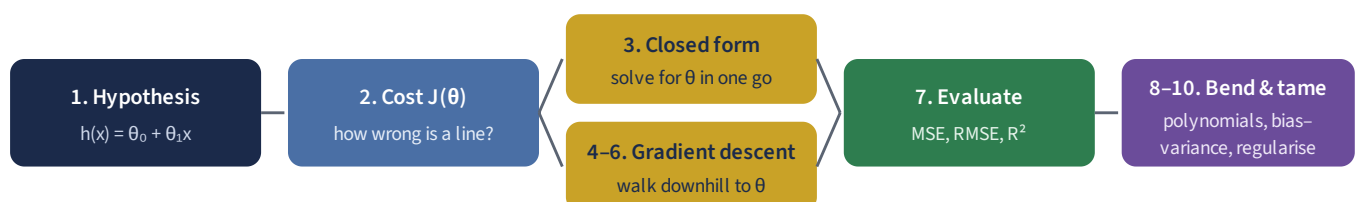
### STOP AND THINK

Arjun's old rule of thumb was "price = 6 lakh − 0.02 lakh per thousand km". Write it as  $h_{\theta}(x)$  with  $x$  in thousands of km. What would the rule predict for a car with 80,000 km? Is the sign of  $\theta_1$  sensible?

### OUR ANSWER

$h_{\theta}(x) = 6 - 0.02x$ , so  $\theta_0 = 6$ ,  $\theta_1 = -0.02$ . For  $x = 80$ :  $6 - 1.6 = 4.4$  lakh. A negative slope is exactly right — more kilometres, lower price. Notice the intercept "6 lakh at zero km" is a *brand-new* car's price under this rule; intercepts are often only meaningful as anchors, not as real predictions.

## How the chapters fit together



The road map of Sessions 3–4. Two routes lead from the cost to the answer: solve it (closed form) or walk to it (gradient descent).

**IN ONE BREATH**

A linear model is  $h_{\theta}(\mathbf{x}) = \boldsymbol{\theta}^T \mathbf{x}$  with  $x_0 = 1$ . Its parameters  $\theta_j$  are learnt; its hyper-parameters are chosen by us. The whole of regression is: define what "best" means, then find the  $\boldsymbol{\theta}$  that achieves it.

## CHAPTER 2

# What Does "Best" Mean? The Cost Function

*Residuals, squared errors, the cost  $J(\theta)$  — and why it is a bowl.*

## STORY

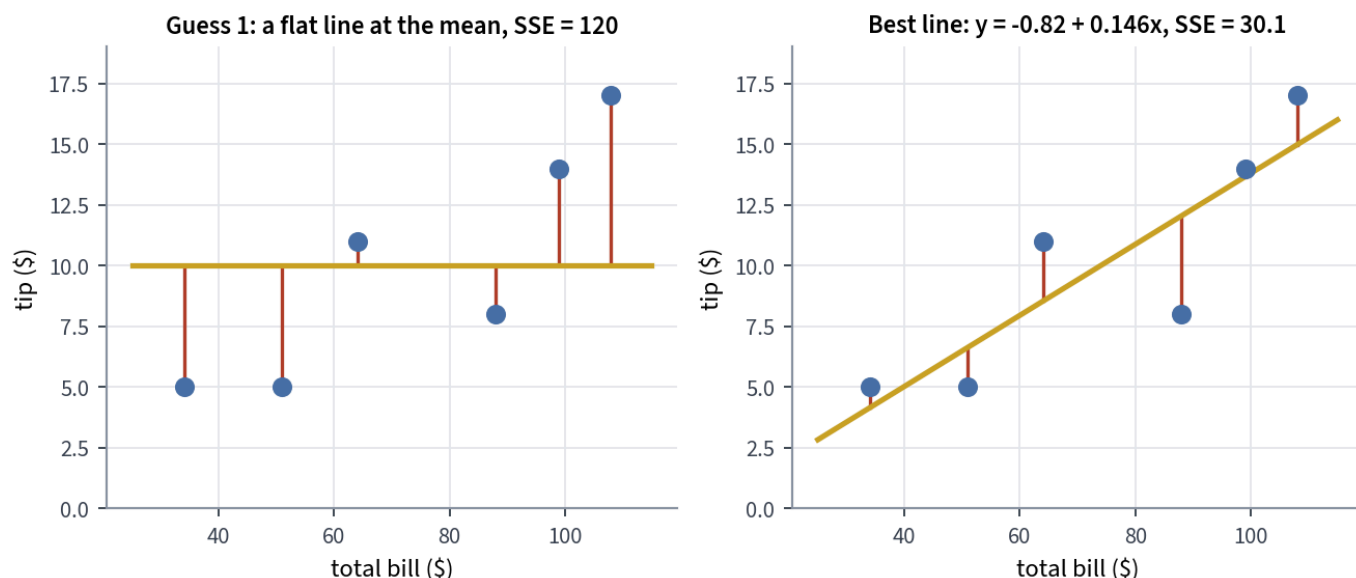
Meera draws two lines and Arjun asks which is better. "This one looks closer." "Closer *how much*? Give me a number. If you cannot score a line, you cannot compare two lines, and you certainly cannot let a computer choose among a million of them."

## Residuals: how far each point is from the line

The slides use a small tips dataset — six restaurant bills and the tip left on each. Start with the laziest possible model: ignore the bill and always predict the average tip,  $y = \theta_0$ . The mean tip is \$10.

| Meal                        | Bill $x$ (\$) | Tip $y$ (\$) | Prediction $\hat{y} = 10$ | Residual $y - \hat{y}$ | Residual <sup>2</sup> |
|-----------------------------|---------------|--------------|---------------------------|------------------------|-----------------------|
| 1                           | 34            | 5            | 10                        | −5                     | 25                    |
| 2                           | 108           | 17           | 10                        | +7                     | 49                    |
| 3                           | 64            | 11           | 10                        | +1                     | 1                     |
| 4                           | 88            | 8            | 10                        | −2                     | 4                     |
| 5                           | 99            | 14           | 10                        | +4                     | 16                    |
| 6                           | 51            | 5            | 10                        | −5                     | 25                    |
| Sum of squared errors (SSE) |               |              |                           |                        | 120                   |

Each **residual** (error) is the vertical distance from a point to the line. We square them before adding so that positive and negative misses cannot cancel, and so that big misses are punished much more than small ones. The total is the **sum of squared errors**, here 120.



Left: the flat "mean" line and its residuals (red). Right: the best-fitting line from Chapter 3 — the residuals shrink and the SSE drops from 120 to about 30.

## The cost function $J(\theta)$

### BIG IDEA

Score a line by the average squared residual. Smaller score = better line. For a hypothesis  $h_\theta$  and  $n$  training examples,

$$J(\theta) = \frac{1}{2n} \sum_{i=1}^n \left( h_\theta(x^{(i)}) - y^{(i)} \right)^2.$$

**Least squares** means: choose  $\theta$  to minimise  $J$ . The best line is  $\min_{\theta} J(\theta)$ .

### CAREFUL — THAT $\frac{1}{2n}$

Dividing by  $n$  turns the sum into an average, so the cost does not grow just because we have more data. The extra  $\frac{1}{2}$  is pure convenience: when we differentiate a square the 2 comes down and cancels it. It does not change *which*  $\theta$  is best — the minimiser of  $J$  is the same as the minimiser of the plain SSE. Different books use SSE,  $\frac{1}{n}$  SSE (the MSE) or  $\frac{1}{2n}$  SSE; in an exam use the version the question gives, and say which you used.

### THINK OF IT LIKE

Golf. Each hole you play, you count how far from the pin you land — but squared, so one wild shot into the lake hurts far more than six shots that land a metre short. Your "cost" for a round is the average of those squared misses. The best swing is the one with the lowest cost, and practising is just trying to lower it.

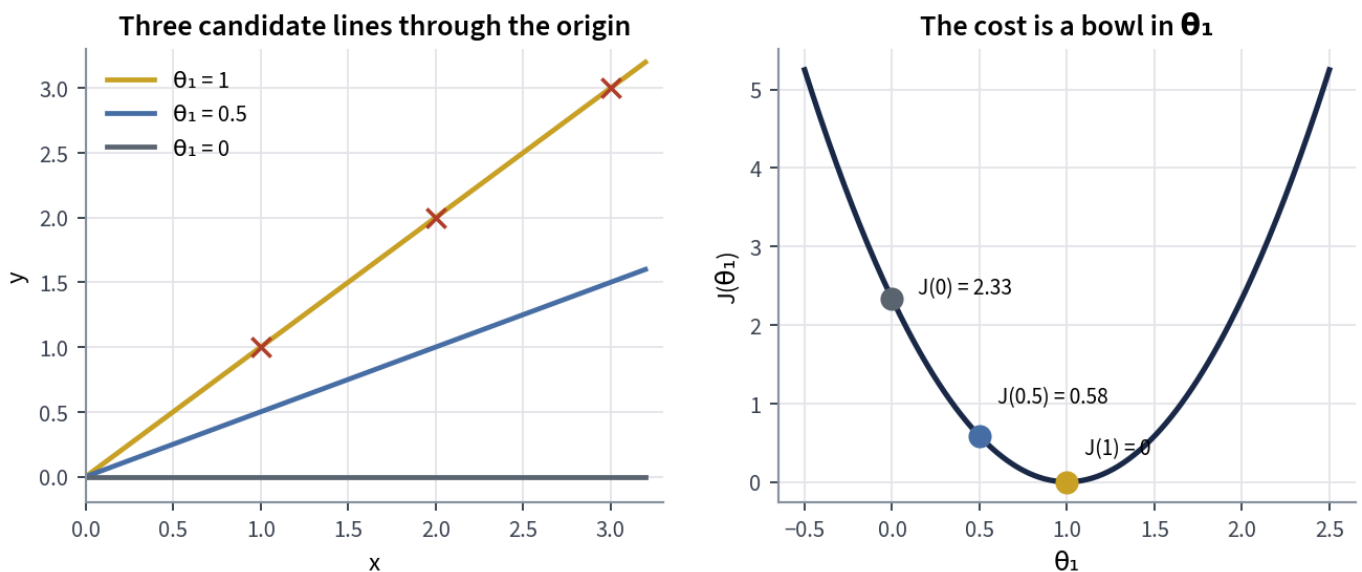
## Seeing $J$ : the bowl

To picture  $J$ , the slides use Andrew Ng's three points  $(1, 1)$ ,  $(2, 2)$ ,  $(3, 3)$  and a line through the origin,  $h_\theta(x) = \theta_1 x$ . Then  $J$  is a function of one number,  $\theta_1$ , and we can evaluate it by hand.

### WORKED EXAMPLE

- 1  $\theta_1 = 1$ : predictions 1, 2, 3 — all exact.  $J(1) = \frac{1}{2 \cdot 3}(0 + 0 + 0) = 0$ .
- 2  $\theta_1 = 0.5$ : predictions 0.5, 1, 1.5.  $J(0.5) = \frac{1}{6}[(0.5 - 1)^2 + (1 - 2)^2 + (1.5 - 3)^2] = \frac{1}{6}[0.25 + 1 + 2.25] = \frac{3.5}{6} \approx 0.58$ .
- 3  $\theta_1 = 0$ : predictions 0, 0, 0.  $J(0) = \frac{1}{6}[1 + 4 + 9] = \frac{14}{6} \approx 2.33$ .

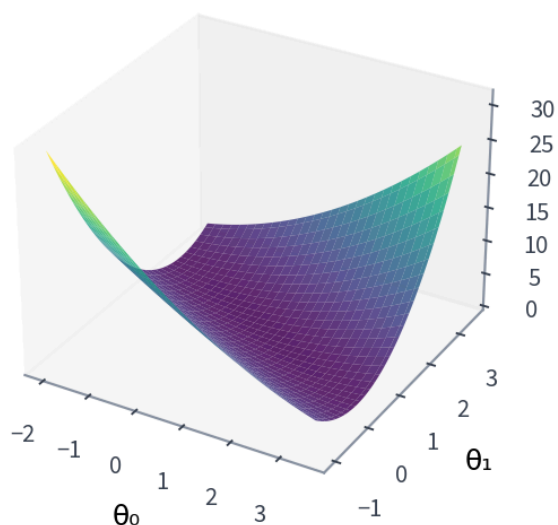
By symmetry  $J(1.5) = 0.58$  and  $J(2) = 2.33$  too. Plot  $J$  against  $\theta_1$  and you get a bowl with its bottom at  $\theta_1 = 1$ .



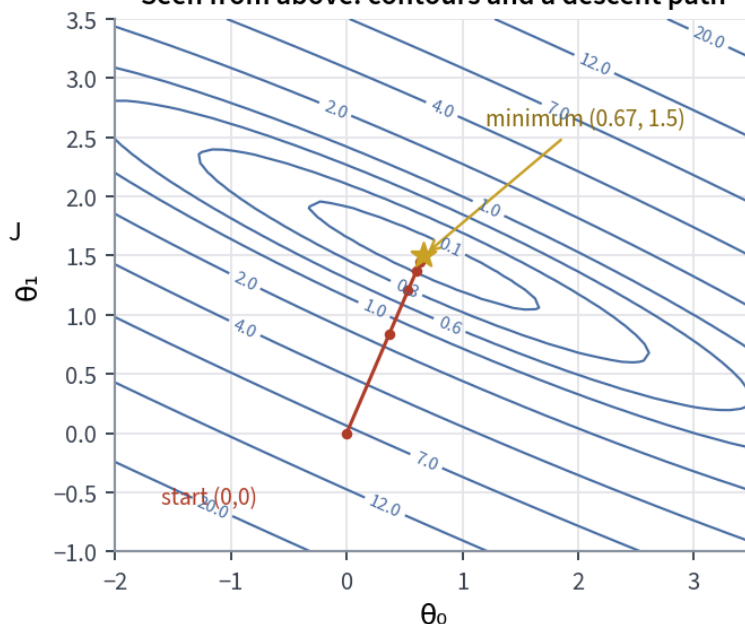
Left: each  $\theta_1$  is a different line. Right: each line is one point on the cost curve. Finding the best line = finding the bottom of the bowl.

With two parameters  $(\theta_0, \theta_1)$  the same thing happens in 3-D:  $J$  is a bowl-shaped surface, and seen from above its level curves are ellipses.

A bowl in two parameters



Seen from above: contours and a descent path



The cost surface for the tiny dataset of Chapter 4 ( $x = 1, 2, 3; y = 2, 4, 5$ ), and its contour map. The red path is gradient descent walking to the bottom.

## Why the bowl matters: convexity

A function is **convex** if the line segment between any two points on its graph lies on or above the graph — a bowl, never a wavy landscape with several dips. The squared-error cost is convex in  $\theta$ . That gives two enormous gifts:

- There is **only one minimum**. Whatever finds a minimum finds *the* minimum. (The slides show a wavy car-price curve with several maxima and minima to make the contrast: that can happen for other cost functions, not for least squares.)
- At the minimum the **slope is zero**:  $\frac{\partial J}{\partial \theta_j} = 0$  for every  $j$ . Chapter 3 solves these equations directly; Chapters 4–6 walk downhill until the slope vanishes.

### STOP AND THINK

Why not use the sum of *absolute* residuals  $\sum |h_\theta(x^{(i)}) - y^{(i)}|$  as the cost? It is also zero for a perfect fit.

### OUR ANSWER

It is a legitimate cost (it is the MAE of Chapter 7 and is more robust to outliers). But the absolute value has a sharp corner at zero, so "set the derivative to zero" has no clean closed-form solution and gradient descent stumbles at the corner. The square is smooth everywhere, differentiates to something linear in  $\theta$ , and gives us Chapter 3's formula. Convenience won.

### IN ONE BREATH

Residual = actual — predicted. Cost  $J(\theta) = \frac{1}{2n} \sum (\text{residual})^2$ . Best line = smallest  $J$ .  $J$  is a convex bowl, so it has one bottom, and at the bottom every partial derivative is zero.

## CHAPTER 3

# Route 1: Solve It — The Closed-Form Solution

*Set the derivatives to zero and solve. First for one input (two formulas), then for any number of inputs (the normal equation).*

## STORY

"Do I have to try a million lines?" Meera asks. Arjun shakes his head. "For a bowl you can go straight to the bottom. Calculus tells you where the slope is zero. Two formulas — and for many inputs, one matrix equation."

## Part A — one input: the two formulas

### MATHS, STEP BY STEP — DERIVING $\theta_0$ AND $\theta_1$

Cost for  $h_\theta(x) = \theta_0 + \theta_1 x$  (we use the plain SSE; the constant in front changes nothing):

$$J = \sum_{i=1}^n (\theta_0 + \theta_1 x_i - y_i)^2.$$

- 1 Differentiate with respect to  $\theta_0$**  (chain rule: bring the 2 down, times the inner derivative, which is 1) and set to zero:

$$\frac{\partial J}{\partial \theta_0} = 2 \sum_i (\theta_0 + \theta_1 x_i - y_i) = 0 \Rightarrow n\theta_0 + \theta_1 \sum_i x_i = \sum_i y_i.$$

Divide by  $n$ :  $\theta_0 + \theta_1 \bar{x} = \bar{y}$ , i.e.  $\boxed{\theta_0 = \bar{y} - \theta_1 \bar{x}}$ . The best line passes through the point of means  $(\bar{x}, \bar{y})$ .

- 2 Differentiate with respect to  $\theta_1$**  (inner derivative is  $x_i$ ):

$$\frac{\partial J}{\partial \theta_1} = 2 \sum_i (\theta_0 + \theta_1 x_i - y_i) x_i = 0.$$

- 3 Substitute  $\theta_0 = \bar{y} - \theta_1 \bar{x}$**  into step 2:

$$\sum_i (\bar{y} - \theta_1 \bar{x} + \theta_1 x_i - y_i) x_i = 0 \Rightarrow \sum_i (\theta_1 (x_i - \bar{x}) - (y_i - \bar{y})) x_i = 0.$$

- 4 A small trick:**  $\sum_i (x_i - \bar{x}) c = 0$  for any constant  $c$ , so we may replace  $x_i$  by  $(x_i - \bar{x})$  inside the sum without changing it. Then

$$\theta_1 \sum_i (x_i - \bar{x})^2 = \sum_i (x_i - \bar{x})(y_i - \bar{y}) \Rightarrow \boxed{\theta_1 = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sum_i (x_i - \bar{x})^2}}.$$

Read  $\theta_1$  as "how  $x$  and  $y$  move together, divided by how much  $x$  moves on its own" — covariance over variance.

**WORKED EXAMPLE — THE TIPS DATA (SLIDE)**

$$\bar{x} = \frac{34+108+64+88+99+51}{6} = \frac{444}{6} = 74, \quad \bar{y} = \frac{5+17+11+8+14+5}{6} = \frac{60}{6} = 10.$$

| Meal | $x_i$ | $y_i$ | $x_i - \bar{x}$ | $y_i - \bar{y}$ | $(x_i - \bar{x})(y_i - \bar{y})$ | $(x_i - \bar{x})^2$ |
|------|-------|-------|-----------------|-----------------|----------------------------------|---------------------|
| 1    | 34    | 5     | -40             | -5              | 200                              | 1600                |
| 2    | 108   | 17    | 34              | 7               | 238                              | 1156                |
| 3    | 64    | 11    | -10             | 1               | -10                              | 100                 |
| 4    | 88    | 8     | 14              | -2              | -28                              | 196                 |
| 5    | 99    | 14    | 25              | 4               | 100                              | 625                 |
| 6    | 51    | 5     | -23             | -5              | 115                              | 529                 |
| Sums |       |       |                 |                 | <b>615</b>                       | <b>4206</b>         |

$$\theta_1 = \frac{615}{4206} \approx 0.1462, \quad \theta_0 = \bar{y} - \theta_1 \bar{x} = 10 - 0.1462 \times 74 = 10 - 10.8188 = -0.8188.$$

Best line:  $\hat{y} = 0.1462x - 0.8188$  — about 14.6 cents of tip per dollar of bill. Predictions:  $\hat{y}_1 = 0.1462(34) - 0.8188 = 4.15$ ,  $\hat{y}_2 = 14.97$ ,  $\hat{y}_3 = 8.54$ ,  $\hat{y}_4 = 12.05$ ,  $\hat{y}_5 = 13.65$ ,  $\hat{y}_6 = 6.64$ . The SSE falls from 120 (flat line) to  $\approx 30.1$ .

**Part B — any number of inputs: vectorise**

With  $d$  inputs, we would have  $d + 1$  equations of the kind above. Writing them out one by one is hopeless; writing them as one matrix equation is easy. The slides use the years-of-experience / salary data:

| Years $x$ | Salary $y$ (lakh) |
|-----------|-------------------|
| 1         | 2                 |
| 2         | 3                 |
| 3         | 4                 |
| 4         | 5                 |
| 5         | 6                 |

**PLAIN MATHS (ONE EXAMPLE)**

Prediction for example  $i$  with the "1" feature:

$$h(\mathbf{x}^{(i)}) = \sum_{j=0}^d \theta_j x_j^{(i)} = \boldsymbol{\theta}^T \mathbf{x}^{(i)}$$

Here  $\boldsymbol{\theta} = \begin{bmatrix} \theta_0 \\ \theta_1 \end{bmatrix}$ ,  $\mathbf{x}^{(i)} = \begin{bmatrix} 1 \\ x_1^{(i)} \end{bmatrix}$ .

**MATRIX FORM (ALL EXAMPLES AT ONCE)**

Stack the examples as rows of a **design matrix**  $\mathbf{X}$  ( $n \times (d + 1)$ ) and the targets in  $\mathbf{y}$  ( $n \times 1$ ):

$$\mathbf{X} = \begin{bmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \\ 1 & 4 \\ 1 & 5 \end{bmatrix}, \quad \mathbf{y} = \begin{bmatrix} 2 \\ 3 \\ 4 \\ 5 \\ 6 \end{bmatrix}, \quad h_{\boldsymbol{\theta}}(\mathbf{X}) = \mathbf{X}\boldsymbol{\theta}$$

One multiplication gives all  $n$  predictions.

Benefits of vectorisation, from the slides: **more compact equations** and **faster code**, because matrix libraries are heavily optimised.

### MATHS, STEP BY STEP – THE NORMAL EQUATION

The cost in matrix form. The residual vector is  $\mathbf{X}\boldsymbol{\theta} - \mathbf{y}$ , and a sum of squares is a dot product of a vector with itself:

$$J(\boldsymbol{\theta}) = \frac{1}{2n} \sum_{i=1}^n (\boldsymbol{\theta}^\top \mathbf{x}^{(i)} - y^{(i)})^2 = \frac{1}{2n} (\mathbf{X}\boldsymbol{\theta} - \mathbf{y})^\top (\mathbf{X}\boldsymbol{\theta} - \mathbf{y}).$$

- 1 **Expand** the product (dropping the constant  $\frac{1}{2n}$ , which does not move the minimum;  $\propto$  means "up to that constant"):

$$J \propto \boldsymbol{\theta}^\top \mathbf{X}^\top \mathbf{X} \boldsymbol{\theta} - \mathbf{y}^\top \mathbf{X} \boldsymbol{\theta} - \boldsymbol{\theta}^\top \mathbf{X}^\top \mathbf{y} + \mathbf{y}^\top \mathbf{y}.$$

- 2 **Merge the middle terms.**  $\mathbf{y}^\top \mathbf{X} \boldsymbol{\theta}$  is a  $1 \times 1$  number, and a number equals its own transpose:  $(\mathbf{y}^\top \mathbf{X} \boldsymbol{\theta})^\top = \boldsymbol{\theta}^\top \mathbf{X}^\top \mathbf{y}$ . So the two middle terms are equal:

$$J \propto \boldsymbol{\theta}^\top \mathbf{X}^\top \mathbf{X} \boldsymbol{\theta} - 2 \boldsymbol{\theta}^\top \mathbf{X}^\top \mathbf{y} + \mathbf{y}^\top \mathbf{y}.$$

- 3 **Differentiate with respect to the vector  $\boldsymbol{\theta}$ .** Two rules, the matrix cousins of  $\frac{d}{d\theta}(a\theta^2) = 2a\theta$  and  $\frac{d}{d\theta}(b\theta) = b$ :

$$\frac{\partial}{\partial \boldsymbol{\theta}} (\boldsymbol{\theta}^\top \mathbf{A} \boldsymbol{\theta}) = 2\mathbf{A}\boldsymbol{\theta} \quad (\mathbf{A} \text{ symmetric}), \quad \frac{\partial}{\partial \boldsymbol{\theta}} (\boldsymbol{\theta}^\top \mathbf{b}) = \mathbf{b}.$$

With  $\mathbf{A} = \mathbf{X}^\top \mathbf{X}$  (symmetric) and  $\mathbf{b} = \mathbf{X}^\top \mathbf{y}$ ; the last term has no  $\boldsymbol{\theta}$  and vanishes:

$$\frac{\partial J}{\partial \boldsymbol{\theta}} \propto 2\mathbf{X}^\top \mathbf{X} \boldsymbol{\theta} - 2\mathbf{X}^\top \mathbf{y}.$$

- 4 **Set to zero and solve:**

$$\mathbf{X}^\top \mathbf{X} \boldsymbol{\theta} = \mathbf{X}^\top \mathbf{y} \Rightarrow \boxed{\boldsymbol{\theta} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}}$$

This is the **normal equation**. It contains Part A as the special case  $d = 1$ .

### WORKED EXAMPLE – PROBLEM TYPE 1 (SLIDE)

- 1 
$$\mathbf{X}^\top \mathbf{X} = \begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 1 & 2 & 3 & 4 & 5 \end{bmatrix} \begin{bmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \\ 1 & 4 \\ 1 & 5 \end{bmatrix} = \begin{bmatrix} 5 & 15 \\ 15 & 55 \end{bmatrix}$$
 (top-left:  $n = 5$ ; off-diagonal:  $\sum x = 15$ ; bottom-right:  $\sum x^2 = 55$ ).

- 2 Inverse of a  $2 \times 2$ :  $\begin{bmatrix} a & b \\ c & d \end{bmatrix}^{-1} = \frac{1}{ad-bc} \begin{bmatrix} d & -b \\ -c & a \end{bmatrix}$ . Determinant =  $5 \cdot 55 - 15 \cdot 15 = 275 - 225 = 50$ , so

$$(\mathbf{X}^\top \mathbf{X})^{-1} = \frac{1}{50} \begin{bmatrix} 55 & -15 \\ -15 & 5 \end{bmatrix} = \begin{bmatrix} 1.1 & -0.3 \\ -0.3 & 0.1 \end{bmatrix}.$$

$$\textcircled{3} \quad \mathbf{X}^T \mathbf{y} = \begin{bmatrix} 2 + 3 + 4 + 5 + 6 \\ 1(2) + 2(3) + 3(4) + 4(5) + 5(6) \end{bmatrix} = \begin{bmatrix} 20 \\ 70 \end{bmatrix}.$$

$$\textcircled{4} \quad \boldsymbol{\theta} = \begin{bmatrix} 1.1 & -0.3 \\ -0.3 & 0.1 \end{bmatrix} \begin{bmatrix} 20 \\ 70 \end{bmatrix} = \begin{bmatrix} 22 - 21 \\ -6 + 7 \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}.$$

So salary =  $1 + 1 \times \text{years}$  — which you can verify fits every row exactly ( $1 + 1 = 2$ ,  $1 + 2 = 3$ , ...).

## When $\mathbf{X}^T \mathbf{X}$ cannot be inverted

The inverse fails when  $\mathbf{X}^T \mathbf{X}$  is **singular**. This happens when two features are exact copies or linear combinations of each other (price in rupees and price in lakhs), or when there are more features than examples ( $d > n$ ). Remedies from the slides: use the **pseudo-inverse** (`numpy.linalg.pinv`); **remove redundant** (linearly dependent) features; **remove extra features** so that  $d \leq n$ . Regularisation (Chapter 10) also fixes it, by adding  $\lambda \mathbf{I}$  to  $\mathbf{X}^T \mathbf{X}$ .

### EXAM TIP

"Fit a linear regression using the closed form" = build  $\mathbf{X}$  with a column of ones, compute  $\mathbf{X}^T \mathbf{X}$  (write the three sums  $n$ ,  $\sum x$ ,  $\sum x^2$  explicitly), invert with the  $2 \times 2$  formula, compute  $\mathbf{X}^T \mathbf{y}$  ( $\sum y$ ,  $\sum xy$ ), multiply. Check your answer by plugging one row back in.

### IN ONE BREATH

One input:  $\theta_1 = \frac{\sum (x - \bar{x})(y - \bar{y})}{\sum (x - \bar{x})^2}$ ,  $\theta_0 = \bar{y} - \theta_1 \bar{x}$ . Any inputs:  $\boldsymbol{\theta} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$ . Both come from "differentiate  $J$ , set to zero, solve".

## CHAPTER 4

# Route 2: Walk Downhill — Gradient Descent

*The update rule, where it comes from, why it always goes downhill, and a full numerical run.*

## STORY

With 40,000 cars and 30 features, Meera's  $\mathbf{X}^T\mathbf{X}$  inverts fine — but Arjun asks what she will do when the catalogue has 40 million cars and 3,000 features. "Inverting a  $3000 \times 3000$  matrix every night? And what about listings that arrive every minute?" There is another route to the bottom of the bowl: start anywhere and keep stepping downhill.

## The basic search procedure

### BIG IDEA

Choose an initial  $\theta$ . Until you reach a minimum, choose a new  $\theta$  that reduces  $J(\theta)$ . Because least squares is convex, there are no local minima to get trapped in — any downhill walk ends at the global bottom.

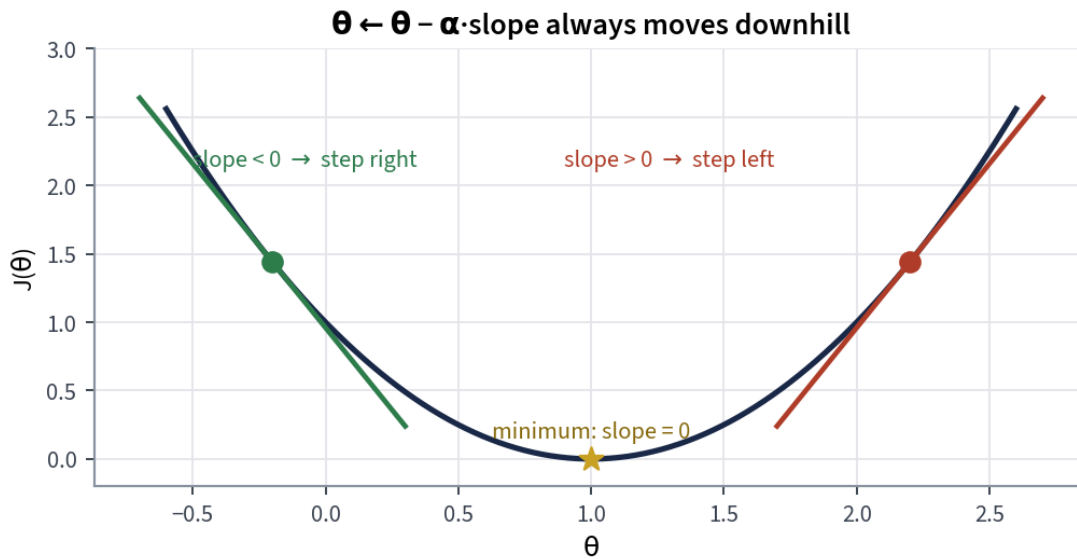
### THINK OF IT LIKE

Walking down a hill in thick fog. You cannot see the valley, but you can feel the slope under your feet. Take a step in the steepest downhill direction, feel the slope again, step again. The **gradient** is the slope under your feet; the **learning rate** is your stride length.

## The update rule

$$\theta_j \leftarrow \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta) \quad \text{simultaneously for } j = 0, 1, \dots, d.$$

|                                        |                                                                                                                                                                                                  |
|----------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $\alpha$                               | The <b>learning rate</b> — a small positive number like 0.05. A hyper-parameter.                                                                                                                 |
| $\frac{\partial J}{\partial \theta_j}$ | The slope of the cost in the direction of knob $j$ . Positive slope means "increasing $\theta_j$ increases $J$ ".                                                                                |
| <b>Simultaneous</b>                    | Compute <i>all</i> the slopes using the <i>old</i> $\theta$ , then update all knobs together. Do not update $\theta_0$ and then use the new $\theta_0$ when computing the slope for $\theta_1$ . |



Why the minus sign works. Right of the minimum the slope is positive, so  $\theta - \alpha \cdot (\text{positive})$  moves left. Left of the minimum the slope is negative, so  $\theta - \alpha \cdot (\text{negative})$  moves right. Either way, downhill. At the bottom the slope is zero and  $\theta$  stops moving — that is convergence.

### MATHS, STEP BY STEP — THE GRADIENT FOR LINEAR REGRESSION

Start from the cost and differentiate with respect to one parameter  $\theta_j$ :

- 1 
$$\frac{\partial}{\partial \theta_j} J(\theta) = \frac{\partial}{\partial \theta_j} \frac{1}{2n} \sum_{i=1}^n \left( h_{\theta}(\mathbf{x}^{(i)}) - y^{(i)} \right)^2$$
- 2 Write out the hypothesis:  $h_{\theta}(\mathbf{x}^{(i)}) = \sum_{k=0}^d \theta_k x_k^{(i)}$ .  

$$= \frac{\partial}{\partial \theta_j} \frac{1}{2n} \sum_{i=1}^n \left( \sum_{k=0}^d \theta_k x_k^{(i)} - y^{(i)} \right)^2$$
- 3 Chain rule: derivative of  $(\text{stuff})^2$  is  $2 \cdot (\text{stuff}) \cdot (\text{derivative of stuff})$ . The 2 cancels the  $\frac{1}{2}$ :  

$$= \frac{1}{n} \sum_{i=1}^n \left( \sum_k \theta_k x_k^{(i)} - y^{(i)} \right) \times \frac{\partial}{\partial \theta_j} \left( \sum_k \theta_k x_k^{(i)} - y^{(i)} \right)$$
- 4 Inside the second bracket only the  $k = j$  term contains  $\theta_j$ , and its derivative is  $x_j^{(i)}$ :  

$$\frac{\partial J}{\partial \theta_j} = \frac{1}{n} \sum_{i=1}^n \left( h_{\theta}(\mathbf{x}^{(i)}) - y^{(i)} \right) x_j^{(i)}$$

In words: **the slope for knob  $j$  is the average of (error  $\times$  feature  $j$ )**. For  $j = 0$ ,  $x_0^{(i)} = 1$ , so it is just the average error.

### PLAIN MATHS — GRADIENT DESCENT FOR LINEAR REGRESSION

Repeat until convergence:

$$\theta_j \leftarrow \theta_j - \alpha \frac{1}{n} \sum_{i=1}^n \left( h_{\theta}(\mathbf{x}^{(i)}) - y^{(i)} \right) x_j^{(i)}$$

for every  $j$  simultaneously. Compute all  $h_{\theta}(\mathbf{x}^{(i)})$  once at the start of the iteration and reuse them.

### MATRIX FORM — WHAT THE COMPUTER RUNS

$$\mathbf{e} = \mathbf{X}\boldsymbol{\theta} - \mathbf{y} \quad (n \times 1 \text{ error vector})$$

$$\nabla J = \frac{1}{n} \mathbf{X}^T \mathbf{e} \quad ((d+1) \times 1)$$

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \alpha \nabla J$$

Three lines, no loops over  $j$ .  $\mathbf{X}^T \mathbf{e}$  computes "sum of error  $\times$  feature  $j$ " for all  $j$  at once.

### WORKED EXAMPLE — THE SLIDE'S TINY DATASET, ONE FULL ITERATION

Predict house price from size. Data: (1, 2), (2, 4), (3, 5). Hyper-parameters  $\eta = 0.1$  (the slide's name for  $\alpha$ ),  $N = 3$ . Start at  $\mathbf{w}^{(0)} = (0, 0)^T$ .

1 **Set up.**  $\mathbf{X} = \begin{bmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \end{bmatrix}$ ,  $\mathbf{y} = \begin{bmatrix} 2 \\ 4 \\ 5 \end{bmatrix}$ ,  $\mathbf{w} = \begin{bmatrix} w_0 \\ w_1 \end{bmatrix}$ .

2 **Predict.**  $\hat{\mathbf{y}}^{(0)} = \mathbf{X}\mathbf{w}^{(0)} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$ .

3 **Initial loss.**  $J = \frac{1}{2N} \|\mathbf{X}\mathbf{w} - \mathbf{y}\|^2 = \frac{1}{6} [(0-2)^2 + (0-4)^2 + (0-5)^2] = \frac{1}{6} (4 + 16 + 25) = \frac{45}{6} = 7.5$ .

4 **Errors.**  $\mathbf{e}^{(0)} = \hat{\mathbf{y}}^{(0)} - \mathbf{y} = \begin{bmatrix} -2 \\ -4 \\ -5 \end{bmatrix}$ .

5 **Gradient.**  $\nabla J = \frac{1}{N} \mathbf{X}^T \mathbf{e} = \frac{1}{3} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 2 & 3 \end{bmatrix} \begin{bmatrix} -2 \\ -4 \\ -5 \end{bmatrix} = \frac{1}{3} \begin{bmatrix} -2 - 4 - 5 \\ -2 - 8 - 15 \end{bmatrix} = \frac{1}{3} \begin{bmatrix} -11 \\ -25 \end{bmatrix} = \begin{bmatrix} -3.67 \\ -8.33 \end{bmatrix}$ .

Plain-maths check for  $w_1$ : average of error  $\times$   $x = \frac{(-2)(1) + (-4)(2) + (-5)(3)}{3} = \frac{-25}{3} \checkmark$

6 **Update.**  $\mathbf{w}^{(1)} = \mathbf{w}^{(0)} - \eta \nabla J = \begin{bmatrix} 0 \\ 0 \end{bmatrix} - 0.1 \begin{bmatrix} -3.67 \\ -8.33 \end{bmatrix} = \begin{bmatrix} 0.367 \\ 0.833 \end{bmatrix}$ .

7 **New predictions and loss.**  $\hat{\mathbf{y}}^{(1)} = \mathbf{X}\mathbf{w}^{(1)} = \begin{bmatrix} 0.367 + 0.833 \\ 0.367 + 1.667 \\ 0.367 + 2.5 \end{bmatrix} = \begin{bmatrix} 1.20 \\ 2.03 \\ 2.87 \end{bmatrix}$ ,  $\mathbf{e}^{(1)} = \begin{bmatrix} -0.80 \\ -1.97 \\ -2.13 \end{bmatrix}$ ,

$$J^{(1)} = \frac{1}{6} (0.64 + 3.87 + 4.55) \approx 1.51.$$

**Progress:** loss 7.5  $\rightarrow$  1.51, an 80% drop in one step. Continue until  $\|\nabla J\| < \epsilon$  or a maximum number of iterations. (Two more steps give  $\mathbf{w} = (0.53, 1.20)$  then  $(0.60, 1.37)$ , loss 0.32; the exact answer from the normal equation is  $(0.667, 1.5)$  with loss  $\approx 0.028$ .)

## The computational graph

The slide's diagram is worth memorising as a story: parameters  $\mathbf{w}$  and data  $\mathbf{X}$  multiply to give  $\hat{\mathbf{y}}$ ; subtract the targets  $\mathbf{y}$  to get the error  $\mathbf{e}$ ; square and sum to get the loss  $J$ . Going *backwards*: the loss's sensitivity to  $\mathbf{e}$  is  $\frac{1}{N}\mathbf{e}$ , and multiplying by  $\mathbf{X}^T$  carries it back to the parameters, giving  $\nabla J$ . This forward-then-backward pattern is exactly what neural networks call back-propagation.

## When to stop

Three practical conventions:

- **Parameters stop moving:**  $\|\boldsymbol{\theta}_{\text{new}} - \boldsymbol{\theta}_{\text{old}}\|_2 < \epsilon$ , where  $\|\mathbf{v}\|_2 = \sqrt{v_1^2 + v_2^2 + \dots}$  is the ordinary (Euclidean,  $L_2$ ) length.
- **Gradient is tiny:**  $\|\nabla J\| < \epsilon$ .
- **A fixed number of iterations**, or the loss on a validation set stops improving.

And the golden diagnostic from the slides: **print  $J(\boldsymbol{\theta})$  every iteration**. It must decrease every time. If it does not, the learning rate is wrong (next chapter).

### EXAM TIP

"Show one iteration of gradient descent" is a fixed recipe: (1) write the hypothesis with the given initial values; (2) compute each prediction and error; (3) compute each gradient as  $\frac{1}{n} \sum (\text{error} \times x_j)$  — show the sum; (4) update every parameter *using the old values*; (5) if asked, compute the new loss. Write the update equations symbolically before the numbers — marks are given for them.

### IN ONE BREATH

$\theta_j \leftarrow \theta_j - \alpha \frac{1}{n} \sum (h_{\theta}(\mathbf{x}^{(i)}) - y^{(i)}) x_j^{(i)}$ , all  $j$  at once; in matrix form  $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \frac{\alpha}{n} \mathbf{X}^T (\mathbf{X}\boldsymbol{\theta} - \mathbf{y})$ . The minus sign guarantees downhill; convexity guarantees the bottom is the answer.

## CHAPTER 5

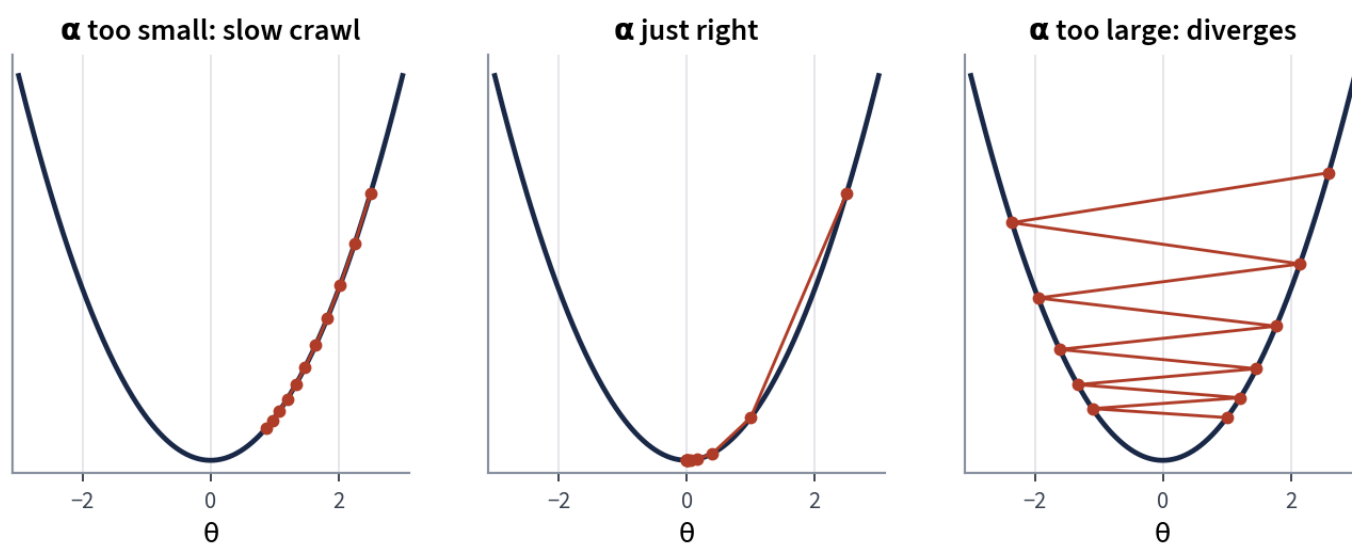
# Making Descent Behave: Learning Rate and Feature Scaling

*Problem Type 2 — reading a convergence plot and diagnosing the hyper-parameters.*

## STORY

Meera's loss goes 7.5, 1.5, 0.3, 0.1, ... — lovely. She tries a bigger stride to go faster: 7.5, 12, 40, 900, ... — the numbers explode. She tries a tiny stride: 7.5, 7.49, 7.48, ... — she will retire before it converges. Then she adds "kilometres" (values in the tens of thousands) next to "age" (values under 15) and descent starts zig-zagging like a drunk. Arjun: "Two knobs you did not know you had."

## Choosing the learning rate $\alpha$



The same bowl, three strides. Small  $\alpha$ : many tiny steps. Right  $\alpha$ : quick, smooth arrival. Large  $\alpha$ : each step overshoots the bottom and lands higher on the other side — divergence.

| $\alpha$ too small                                                                                            | $\alpha$ too large                                                                                                                                          |
|---------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Every step is a crawl. $J$ decreases, but so slowly that you need thousands of iterations. Safe but wasteful. | Steps overshoot the minimum. $J$ may <b>oscillate</b> , <b>fail to converge</b> , or <b>increase every iteration</b> (diverge to infinity). Fast and fatal. |

## THE DIAGNOSTIC (SLIDE)

Plot  $J(\theta)$  against the iteration number. It should **decrease on every iteration**. If it goes up, or bounces, **reduce**  $\alpha$  (try  $\alpha/3$ ). If it decreases but painfully slowly, **increase**  $\alpha$  (try  $3\alpha$ ). Typical grid: 0.001, 0.003, 0.01, 0.03, 0.1, 0.3, 1.

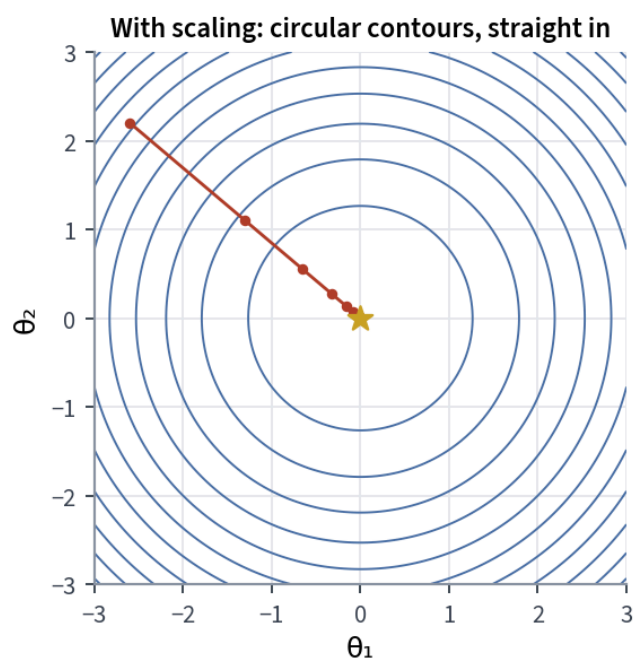
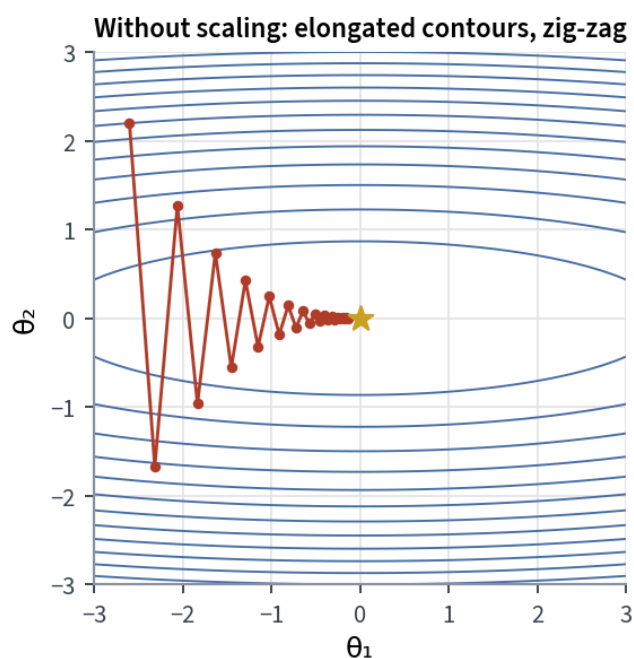
**WORKED EXAMPLE — PROBLEM TYPE 2 (THE THREE-PANEL SLIDE)**

The slide shows three runs of gradient descent on the same data with  $\eta = 0.02, 0.1$  and  $0.5$ , each drawing the fitted line after every iteration. How to read it:

- $\eta = 0.02$ : the lines creep up from the starting position and after ten iterations are still far below the data — **too small, slow convergence**.
- $\eta = 0.1$ : the lines move quickly and settle onto the data — **good**.
- $\eta = 0.5$ : the lines jump above and below the data with growing swings — **too large, diverging**.

Any such question is answered the same way: describe the trend of the fit (or of  $J$ ) across iterations, name the diagnosis, say what to change.

## Feature scaling: why unequal scales cause zig-zag



Left: feature 2 has a much larger scale than feature 1, so the bowl is a long narrow valley; descent bounces between the walls. Right: after scaling, the bowl is round and descent goes straight to the centre.

Gradient descent uses the slope in each parameter direction. When one feature (kilometres,  $\sim 10^5$ ) has a far larger scale than another (age,  $\sim 10$ ), the cost changes much faster along one parameter than the other. The contours become **elongated ellipses**, one direction wants a tiny step and the other wants a big one, and a single  $\alpha$  cannot suit both: it overcorrects across the valley and undercorrects along it — **zig-zagging**.

| Aspect                | With scaling            | Without scaling          |
|-----------------------|-------------------------|--------------------------|
| Contour shape         | Circular                | Elongated                |
| Convergence speed     | Fast                    | Slow                     |
| Gradient descent path | Straight to the minimum | Zig-zagging, inefficient |

The fix is exactly Book 2's feature scaling: **standardisation** ( $\frac{x-\mu}{\sigma}$ , zero mean, unit variance) or **min-max normalisation** (rescale to  $[0, 1]$ ). Every input feature on a similar scale  $\Rightarrow$  circular contours  $\Rightarrow$  direct descent. Remember the rules: fit the scaler on the training set only; do not scale the target; the closed-form solution does not need scaling at all (it is not walking anywhere).

### CAREFUL

Scaling changes the *numbers* of the parameters (a slope per standardised-kilometre is not a slope per kilometre) but not the *predictions*. To interpret coefficients in original units, undo the scaling afterwards.

### STOP AND THINK

The BMI problem in Chapter 6 uses raw features (BMI  $\approx 30$ , diastolic pressure  $\approx 90$ ) with  $\alpha = 0.02$ . Predict what happens if you run it for a second iteration.

### OUR ANSWER

Trouble. The gradient for the pressure weight is a sum of error  $\times$  pressure ( $\sim 90$ ), so one update moves  $\theta_2$  by about 0.2; that changes every prediction by  $\sim 0.2 \times 90 = 18$ , the errors explode, and iteration 2 flings the weights to  $-12$  and  $-35$ . On unscaled features  $\alpha = 0.02$  is far too large — a live demonstration of both lessons of this chapter. Chapter 6 shows the numbers.

### IN ONE BREATH

Watch  $J$  per iteration: rising or bouncing means  $\alpha$  too large, crawling means too small. Put all features on one scale before descent, or the contours stretch and the walk zig-zags.

## CHAPTER 6

# The Exam Problem: Two Predictors, One Iteration

*Problem Type 3 — the BMI / blood-pressure regression worked to the last decimal, plus the batch / stochastic / mini-batch variants.*

## STORY

Arjun sets Meera a warm-up before she touches the car data: a three-patient medical table from the slides. "Two predictors, three rows, one iteration, by hand. If you can do this on paper, the computer version is just the same loop with more rows."

## The problem (slide)

### PROBLEM STATEMENT

Fit a linear regression. Show only the first iteration of gradient descent with learning rate 0.02, if the Relative Risk of Coronary Heart Disease (RR-CHD) is believed to depend linearly on BMI and on Diastolic Pressure. Assume the intercept is 5 and the slope of each predictor is  $-0.03$ .

| Patient | Systolic (mm Hg) | Diastolic (mm Hg) | BMI | Waist (cm) | RR-CHD |
|---------|------------------|-------------------|-----|------------|--------|
| 1       | 140              | 80                | 35  | 100        | 1.81   |
| 2       | 120              | 80                | 25  | 80         | 1.22   |
| 3       | 130              | 100               | 30  | 60         | 1.71   |

Only the shaded columns are used:  $x_1 = \text{BMI}$ ,  $x_2 = \text{Diastolic Pressure}$ ,  $y = \text{RR-CHD}$ .  
Systolic and waist are distractors — the question says "only" BMI and diastolic.

### STEP 1 — IDENTIFY THE EQUATIONS

$$\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2 = 5 - 0.03 \text{ BMI} - 0.03 \text{ DP}$$

with  $n = 3$ ,  $\alpha = 0.02$ , and initial  $\theta = (5, -0.03, -0.03)$ .

### STEP 2 — COST FUNCTION AND UPDATE EQUATIONS

$J = \frac{1}{2n} \sum (\hat{y}^{(i)} - y^{(i)})^2$ , and the three update rules (Chapter 4), written out for this problem:

$$\theta'_0 = \theta_0 - \alpha \cdot \frac{1}{3} \sum_{i=1}^3 (\hat{y}^{(i)} - y^{(i)})$$

$$\theta'_1 = \theta_1 - \alpha \cdot \frac{1}{3} \sum_{i=1}^3 (\hat{y}^{(i)} - y^{(i)}) \text{ BMI}^{(i)}$$

$$\theta'_2 = \theta_2 - \alpha \cdot \frac{1}{3} \sum_{i=1}^3 (\hat{y}^{(i)} - y^{(i)}) \text{DP}^{(i)}$$

### STEP 3 — APPLY THE EQUATIONS

| $i$  | BMI | DP  | $y$  | $\hat{y} = 5 - 0.03 \text{BMI} - 0.03 \text{DP}$ | $e = \hat{y} - y$ | $e \times \text{BMI}$ | $e \times \text{DP}$ |
|------|-----|-----|------|--------------------------------------------------|-------------------|-----------------------|----------------------|
| 1    | 35  | 80  | 1.81 | $5 - 1.05 - 2.40 = 1.55$                         | -0.26             | -9.10                 | -20.80               |
| 2    | 25  | 80  | 1.22 | $5 - 0.75 - 2.40 = 1.85$                         | +0.63             | +15.75                | +50.40               |
| 3    | 30  | 100 | 1.71 | $5 - 0.90 - 3.00 = 1.10$                         | -0.61             | -18.30                | -61.00               |
| Sums |     |     |      |                                                  | -0.24             | -11.65                | -31.40               |

- 1  $\theta'_0 = 5 - 0.02 \cdot \frac{1}{3}(-0.24) = 5 - 0.02(-0.08) = 5 + 0.0016 = \mathbf{5.0016}$
- 2  $\theta'_1 = -0.03 - 0.02 \cdot \frac{1}{3}(-11.65) = -0.03 - 0.02(-3.8833) = -0.03 + 0.07767 = \mathbf{0.0477}$
- 3  $\theta'_2 = -0.03 - 0.02 \cdot \frac{1}{3}(-31.40) = -0.03 - 0.02(-10.4667) = -0.03 + 0.20933 = \mathbf{0.1793}$

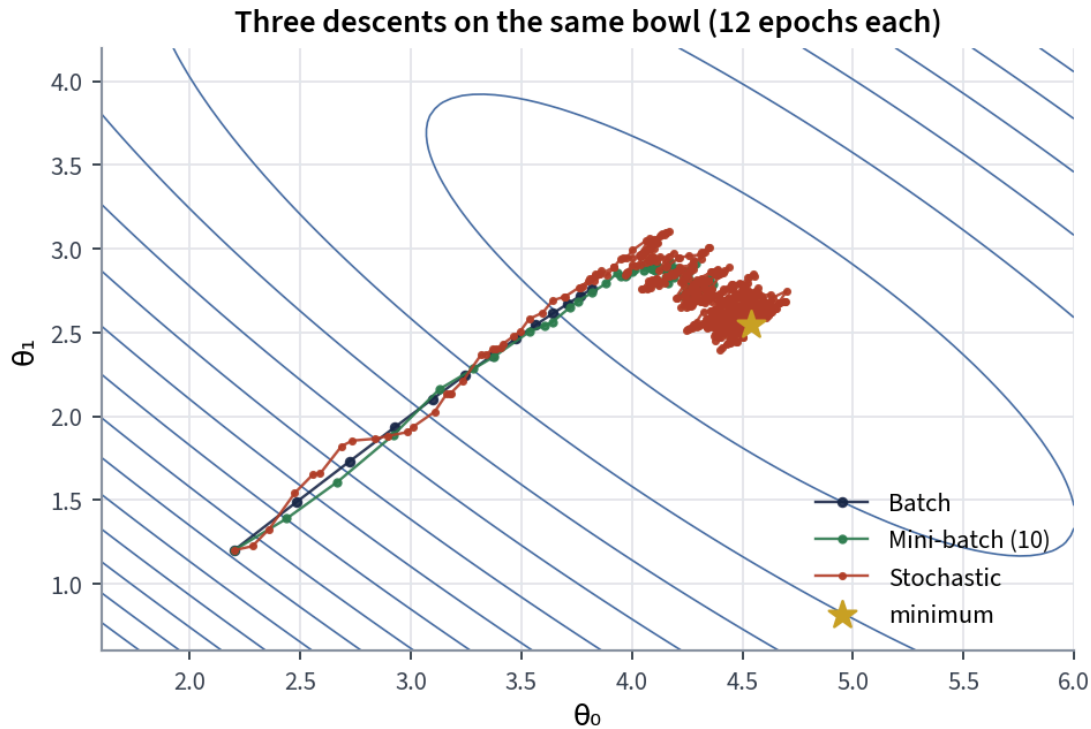
Answer after the first iteration:  $\theta = (5.0016, 0.0477, 0.1793)$ , matching the slide. Initial cost  $J = \frac{1}{6}(0.0676 + 0.3969 + 0.3721) = 0.139$ .

### CAREFUL — WHAT THE SECOND ITERATION TEACHES

The slide's practice version asks for two iterations. Run it and watch: with  $\theta = (5.0016, 0.0477, 0.1793)$  the predictions become 21.0, 20.5, 24.4 (against targets near 1.5), the errors are about +20, and the second update gives  $\theta \approx (4.59, -12.2, -35.5)$ . The cost jumps from 0.14 to over 200. This is not an arithmetic mistake — it is **divergence**:  $\alpha = 0.02$  is far too large for unscaled features whose values are around 30 and 90 (Chapter 5). In an exam, report the numbers, then *say this*: the loss increased, the learning rate is too large for these feature scales, and the fix is to standardise BMI and DP or to reduce  $\alpha$  by a factor of  $\sim 100$ . That interpretation is what the marks are for.

## Batch, stochastic, and mini-batch descent

Everything so far used **all**  $n$  examples to compute each gradient. That is **batch gradient descent**. Two variants change how much data goes into one update.



Batch takes one smooth step per epoch. Stochastic takes  $n$  noisy steps per epoch and ends up jittering around the minimum. Mini-batch is in between.

| Type                    | Data per update      | Updates per epoch | Speed  | Stability | Common use                      |
|-------------------------|----------------------|-------------------|--------|-----------|---------------------------------|
| <b>Batch</b>            | All $n$ examples     | 1                 | Slow   | High      | Small datasets                  |
| <b>Stochastic (SGD)</b> | 1 example            | $n$               | Fast   | Low       | Online learning, streaming data |
| <b>Mini-batch</b>       | A small batch of $m$ | $n/m$             | Medium | Medium    | Deep learning (default)         |

### THE THREE UPDATE RULES SIDE BY SIDE

**Batch** (sum over all  $n$ ):  $\theta_j \leftarrow \theta_j - \alpha \frac{1}{n} \sum_{i=1}^n (h_{\theta}(\mathbf{x}^{(i)}) - y^{(i)}) x_j^{(i)}$

**Mini-batch** (sum over the  $m$  examples of the current batch  $B$ ):  $\theta_j \leftarrow \theta_j - \alpha \frac{1}{m} \sum_{i \in B} (h_{\theta}(\mathbf{x}^{(i)}) - y^{(i)}) x_j^{(i)}$

**Stochastic** (one randomly chosen example  $i$ , no sum, no  $\frac{1}{n}$ ):  $\theta_j \leftarrow \theta_j - \alpha (h_{\theta}(\mathbf{x}^{(i)}) - y^{(i)}) x_j^{(i)}$

An **epoch** is one pass through the training data. SGD shuffles the data each epoch and samples one instance at a time.

**WORKED EXAMPLE — ONE STOCHASTIC STEP**

Tiny data  $(1, 2), (2, 4), (3, 5)$ ,  $\mathbf{w} = (0, 0)$ ,  $\alpha = 0.1$ , and the randomly picked example is the first one,  $x = 1, y = 2$ . Prediction 0, error  $e = 0 - 2 = -2$ . Updates:  $w_0 \leftarrow 0 - 0.1(-2)(1) = 0.2$ ;  $w_1 \leftarrow 0 - 0.1(-2)(1) = 0.2$ . Compare batch's  $(0.367, 0.833)$ : SGD moved in a similar direction using a third of the work — but a different pick would have moved differently.

**THINK OF IT LIKE**

Judging a restaurant. Batch: eat every dish on the menu before forming an opinion — accurate, slow. Stochastic: change your opinion after every single bite — fast, jumpy, but you get somewhere sensible quickly. Mini-batch: revise after each course.

## Closed form versus gradient descent

| Gradient descent                                                     | Closed-form solution                                                                      |
|----------------------------------------------------------------------|-------------------------------------------------------------------------------------------|
| Requires multiple iterations                                         | Non-iterative — one formula                                                               |
| Need to choose $\alpha$ (and scale features)                         | No $\alpha$ , no scaling needed                                                           |
| Works well when $n$ (and $d$ ) are large                             | Slow when $d$ is large: computing $(\mathbf{X}^T \mathbf{X})^{-1}$ costs roughly $O(d^3)$ |
| Supports incremental / online learning (SGD)                         | Must be recomputed from scratch when data changes                                         |
| Works for any differentiable cost (logistic regression, neural nets) | Exists only for least squares                                                             |

**IN ONE BREATH**

Problem Type 3: write the three update equations, tabulate  $\hat{y}, e, e x_1, e x_2$ , sum, multiply by  $\alpha/n$ , update. Batch uses all rows per step, SGD one row (no  $\frac{1}{n}$ ), mini-batch a few. Closed form is exact but  $O(d^3)$ ; descent scales and generalises to other models.

## CHAPTER 7

# How Good Is the Line? Evaluating Regression

*MSE, MAE, RMSE and  $R^2$  — what each one measures and how to compute them by hand.*

## STORY

Meera's price model is trained. Arjun wants one number for the board: "How far off are we, typically?" And a second number for the data team: "How much of the price variation does the model actually explain?" Two different questions, two different metrics.

## Error metrics: how far off, typically?

All three use the residuals  $y_i - \hat{y}_i$  on a set of  $N$  examples — the **test set**, if you want to know about generalisation.

## DEFINITIONS

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2 \quad \text{MAE} = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i| \quad \text{RMSE} = \sqrt{\text{MSE}} = \sqrt{\frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2}$$

**MSE** Mean squared error. What gradient descent minimised (up to the  $\frac{1}{2}$ ). In squared units (\$<sup>2</sup>, lakh<sup>2</sup>) — hard to interpret directly, but punishes big misses heavily.

**RMSE** Root mean squared error. Back in the units of  $y$ . "Typical error, with big misses weighted extra." The most common headline number.

**MAE** Mean absolute error. Also in the units of  $y$ . "Typical error, every miss counted equally." More robust to a few outliers. Always  $\leq$  RMSE.

## WORKED EXAMPLE — THE TIPS LINE ON ITS OWN DATA

| Meal | $y$ | $\hat{y} = 0.1462x - 0.8188$ | $y - \hat{y}$ | $ y - \hat{y} $ | $(y - \hat{y})^2$ |
|------|-----|------------------------------|---------------|-----------------|-------------------|
| 1    | 5   | 4.152                        | 0.848         | 0.848           | 0.719             |
| 2    | 17  | 14.971                       | 2.029         | 2.029           | 4.118             |
| 3    | 11  | 8.538                        | 2.462         | 2.462           | 6.061             |
| 4    | 8   | 12.047                       | -4.047        | 4.047           | 16.377            |
| 5    | 14  | 13.655                       | 0.345         | 0.345           | 0.119             |
| 6    | 5   | 6.637                        | -1.637        | 1.637           | 2.681             |
| Sums |     |                              |               | 11.368          | 30.075            |

$$\text{MSE} = \frac{30.075}{6} = 5.01, \quad \text{RMSE} = \sqrt{5.01} = 2.24, \quad \text{MAE} = \frac{11.368}{6} = 1.89.$$

Typical tip error about 2. *Note*  $RMSE > MAE$  — *mean of squares* pulls the squared version up.

### THINK OF IT LIKE

An archer's scorecard. MAE: average distance of arrows from the bullseye. RMSE: the same, but each arrow's distance is squared first, so one arrow in the next field ruins the score — that is the point, if wild misses are what you fear.

## $R^2$ : how much of the variation is explained?

The slide's picture: the tips vary around their mean  $\bar{y} = 10$ . Some of that variation the line explains (higher bill  $\rightarrow$  higher tip); some it does not (the residuals).  $R^2$  is the explained fraction.

### MATHS, STEP BY STEP

Split each point's deviation from the mean into two pieces:

$$\underbrace{(y_i - \bar{y})}_{\text{total}} = \underbrace{(\hat{y}_i - \bar{y})}_{\text{explained}} + \underbrace{(y_i - \hat{y}_i)}_{\text{unexplained}}$$

Square and sum over all points. For a least-squares line (with intercept) the cross-term sums to zero, so the sums of squares split cleanly:

$$\begin{aligned} SS_{\text{total}} &= \sum_i (y_i - \bar{y})^2, & SS_{\text{explained}} &= \sum_i (\hat{y}_i - \bar{y})^2, & SS_{\text{residual}} &= \sum_i (y_i - \hat{y}_i)^2, & SS_{\text{total}} &= \\ & & & & & & SS_{\text{explained}} + SS_{\text{residual}}. \\ R^2 &= \frac{SS_{\text{explained}}}{SS_{\text{total}}} = 1 - \frac{SS_{\text{residual}}}{SS_{\text{total}}}. \end{aligned}$$

### WORKED EXAMPLE — TIPS

$SS_{\text{total}}$  is the SSE of the flat "mean" line from Chapter 2: 120.  $SS_{\text{residual}}$  is the SSE of the fitted line: 30.075.

$$R^2 = 1 - \frac{30.075}{120} = 1 - 0.2506 = 0.749.$$

The line explains about 75% of the variation in tips; the remaining 25% is due to things not in the model (mood, service, group size). Check:  $SS_{\text{explained}} = \sum (\hat{y}_i - 10)^2 \approx 89.9$ , and  $89.9 + 30.1 \approx 120 \checkmark$ .

$R^2 = 1$  Perfect fit — every residual is zero.

$R^2 = 0$  The model is no better than predicting the mean.

$R^2 < 0$  Possible on *test* data: the model is worse than the mean. A red flag.

**CAREFUL**

$R^2$  is about *fit*, not *error size*: a model can have a high  $R^2$  and still be too inaccurate for the business (if the total variation is huge), or a low  $R^2$  and be perfectly useful (if the target is intrinsically noisy). Report an error metric in real units alongside it. And adding any feature — even a useless one — never lowers training  $R^2$ ; that is why we look at test data and, later, at penalties for complexity.

## Comparing two models (slide)

The slide contrasts Model 1:  $\text{price} = 8.5 + 0.5 \text{ mileage} - 1.5 \text{ mileage}^2$  with Model 2:  $\text{price} = -5.5 + 1.5 \text{ mileage}$ , and asks which is better. The procedure is always the same: **compute each model's predictions on the unseen data** (mileage 7.5  $\rightarrow$  price 9.25; mileage 10  $\rightarrow$  price 6.5; ...), compute the residuals, and compare RMSE (or MAE, or  $R^2$ ) on that unseen set. The training fit is irrelevant to the comparison; only the test error decides. In Chapter 9 the same comparison becomes the underfitting/overfitting diagnosis.

**EXAM TIP**

Tabulate  $y$ ,  $\hat{y}$ , residual,  $|\text{residual}|$ ,  $\text{residual}^2$ . Sum the last two. MAE and MSE are those sums over  $N$ ; RMSE is the square root of MSE. For  $R^2$  you also need  $\sum (y - \bar{y})^2$  — compute  $\bar{y}$  first. Say explicitly which set (train/test) the metrics are on.

**IN ONE BREATH**

MAE: mean  $|\text{error}|$ . MSE: mean  $\text{error}^2$ . RMSE:  $\sqrt{\text{MSE}}$ , in  $y$ 's units.  $R^2 = 1 - \text{SS}_{\text{res}}/\text{SS}_{\text{tot}}$ : fraction of variation explained. Always evaluate on unseen data.

## CHAPTER 8

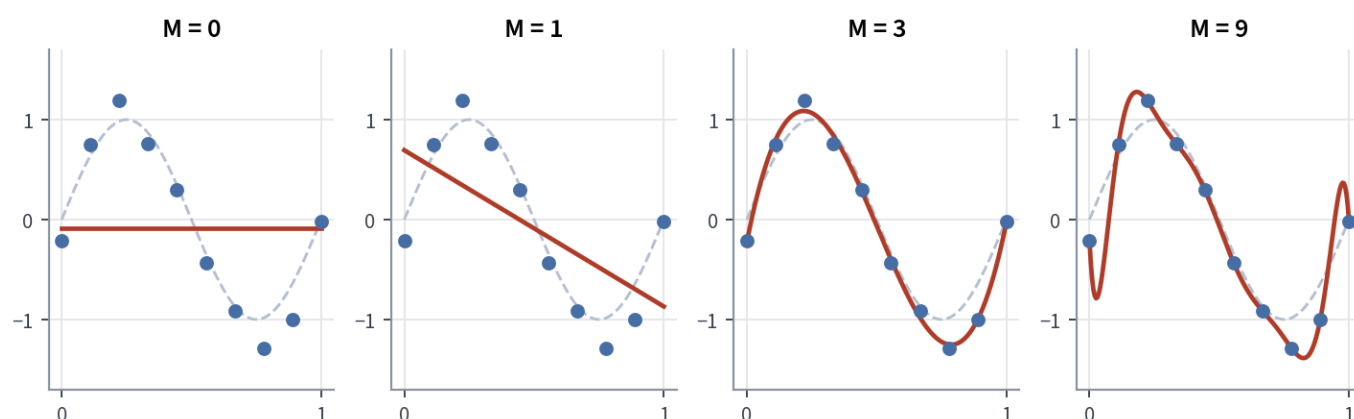
# Bending the Line: Basis Functions and Polynomial Regression

*How a "linear" model fits a curve — linear in the weights, not in the inputs.*

## STORY

Meera's straight line under-prices nearly-new cars and over-prices very old ones: the true drop is steep at first and then flattens — a curve, not a line. "Does that mean I need a completely different algorithm?" "No," says Arjun. "Change the *inputs*, not the algorithm. Feed it kilometres *and* kilometres squared. It is still a linear regression."

## What if the output is a non-linear function of the input?



Ten noisy points from a sine wave (dashed), fitted with polynomials of degree  $M = 0, 1, 3, 9$ .  $M = 3$  captures the shape;  $M = 9$  passes through every point and wiggles wildly in between.

The slide's list of what the inputs  $\mathbf{X}$  of a linear regression can be: original quantitative inputs; **transformations** of them (log, exp, square root, square); **polynomial** terms ( $y = \beta_0 + \beta_1 x + \beta_2 x^2 + \beta_3 x^3$ ); basis expansions; **dummy coding** of categorical inputs (one-hot, Book 2); **interactions** between variables ( $x_3 = x_1 \cdot x_2$ ). "This allows the use of linear regression techniques to fit non-linear datasets."

## Polynomial regression is just multiple regression

### WORKED EXAMPLE — THE SALARY TABLE WITH A SQUARED FEATURE (SLIDE)

| Years $x$ | $x^2$ | Salary $y$ |
|-----------|-------|------------|
| 1         | 1     | 2          |
| 2         | 4     | 3          |
| 3         | 9     | 4          |
| 4         | 16    | 5          |
| 5         | 25    | 6          |

To fit  $y = w_0 + w_1x + w_2x^2$ , simply create the column  $x^2$  and run ordinary multiple regression with two

"features",  $x$  and  $x^2$ . The design matrix becomes  $\mathbf{X} = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 2 & 4 \\ 1 & 3 & 9 \\ 1 & 4 & 16 \\ 1 & 5 & 25 \end{bmatrix}$  and the normal equation or gradient

descent proceeds exactly as before. Nothing in the algorithm knows that column 3 is the square of column 2.

## Linear basis function models — the general form

### MATHS, STEP BY STEP

An  $M$ -th order polynomial in one feature  $x$ :

$$y(x, \mathbf{w}) = w_0 + w_1x + w_2x^2 + \cdots + w_Mx^M = \sum_{j=0}^M w_j x^j.$$

Replace " $x^j$ " by any fixed function  $\phi_j(x)$  — a **basis function** — and you get the general linear basis-function model:

$$y(\mathbf{x}, \mathbf{w}) = \sum_{j=0}^M w_j \phi_j(\mathbf{x}) = \mathbf{w}^T \boldsymbol{\phi}(\mathbf{x}), \quad \phi_0(\mathbf{x}) = 1 \text{ so that } w_0 \text{ is the bias.}$$

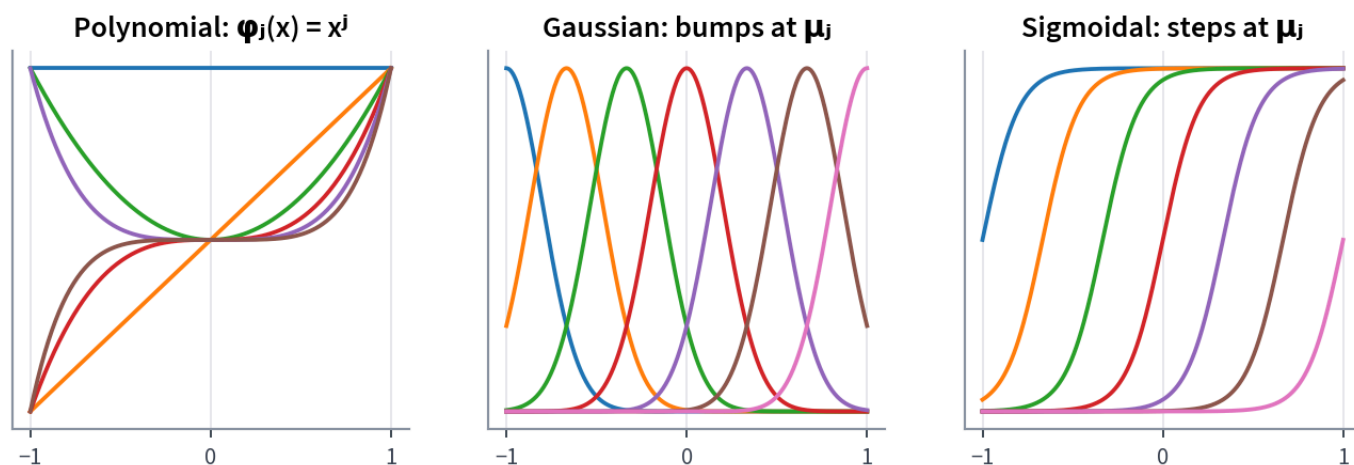
The polynomial case is  $\phi_j(x) = x^j$ . Everything from Chapters 3–4 works unchanged with  $\mathbf{X}$  replaced by the matrix  $\boldsymbol{\Phi}$  whose row  $i$  is  $\boldsymbol{\phi}(\mathbf{x}^{(i)})^T$ :  $\mathbf{w} = (\boldsymbol{\Phi}^T \boldsymbol{\Phi})^{-1} \boldsymbol{\Phi}^T \mathbf{y}$ , and  $\nabla J = \frac{1}{n} \boldsymbol{\Phi}^T (\boldsymbol{\Phi} \mathbf{w} - \mathbf{y})$ .

### BIG IDEA

These are called **linear** models because the prediction is linear in the **weights**  $\mathbf{w}$ , not because the curve is straight. The cost is still a convex bowl in  $\mathbf{w}$ , so the closed form and gradient descent still find the unique minimum — that is the whole reason the trick is so useful.

**THINK OF IT LIKE**

A mixing desk. Each basis function is one pre-recorded track (a straight ramp, a parabola, a bump around  $x = 0.3\dots$ ). The weights are the volume sliders. You never change the tracks — you only move sliders — and yet the mix can be almost any shape. "Linear in  $\mathbf{w}$ " means: sliders only.

**A menu of basis functions**

Three families from the slides. Polynomials are *global*: change  $x$  a little and every basis function changes. Gaussians and sigmoids are *local*: a small change in  $x$  only affects the nearby bumps or steps.

| Family     | Formula                                                                                     | Character                                                                                                                              |
|------------|---------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------|
| Polynomial | $\phi_j(x) = x^j$                                                                           | Global; a small change in $x$ affects all basis functions. Simple, but high degrees explode at the edges.                              |
| Gaussian   | $\phi_j(x) = \exp\left(-\frac{(x - \mu_j)^2}{2s^2}\right)$                                  | Local bumps; $\mu_j$ sets the location, $s$ the width. Only nearby bumps respond to a change in $x$ .                                  |
| Sigmoidal  | $\phi_j(x) = \sigma\left(\frac{x - \mu_j}{s}\right),$<br>$\sigma(a) = \frac{1}{1 + e^{-a}}$ | Local steps; $\mu_j$ sets the location, $s$ the slope. (The same $\sigma$ returns as the heart of logistic regression in Session 5–6.) |

**STOP AND THINK**

Meera fits  $\text{price} = w_0 + w_1 \text{ km} + w_2 \text{ km}^2$  and gets  $w_2 > 0$ . What shape is the curve, and does it make physical sense for very large km?

**OUR ANSWER**

A parabola opening upward: price falls, flattens, then *rises* again for very high km — nonsense (a car with 500,000 km does not become expensive). Polynomials are global and behave badly outside the data range; either restrict predictions to the observed range, or use a basis that flattens out (a log of km, or a sigmoid). This is the "global vs local" point in action.

**IN ONE BREATH**

$y = \mathbf{w}^T \phi(\mathbf{x})$ : transform the inputs with fixed basis functions ( $x^j$ , Gaussians, sigmoids), then do ordinary linear regression on the transformed columns. Linear in  $\mathbf{w} \Rightarrow$  same convex cost, same solvers, curved fits.

## CHAPTER 9

# Bias, Variance, and the Art of Not Overfitting

*Problem Type 4 — reading learning curves and model-fit pictures, and the two easy cures for overfitting.*

## STORY

Encouraged by the curve, Meera tries degree 10. Training RMSE: almost zero. She is delighted. Test RMSE: worse than the straight line. "It memorised the 40,000 cars," says Arjun, "including the one with a scratched bumper that sold cheap. Two words: bias and variance."

## Two kinds of error

### Bias

Error from **wrong assumptions** in the model — how far the model's predictions are from the true relationship, even with unlimited data. **High bias** means the model is **too simple**: a straight line for a curved truth. It **underfits**.

### Variance

Error from **sensitivity to the training data** — how much the predictions would change if we trained on a different sample. **High variance** means the model is **too complex**: a wiggly polynomial that fits every point, noise included. It **overfits**.

## THINK OF IT LIKE

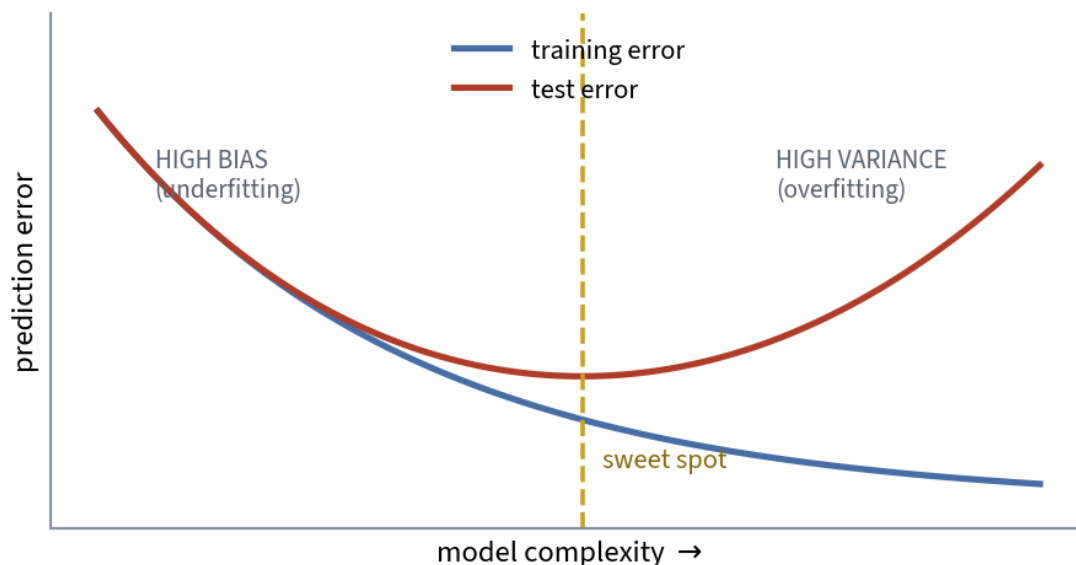
Two weather forecasters. One always says "28 °C" whatever the data — consistent, but wrong in a fixed way: **bias**. The other changes the forecast wildly with every gust of wind — right on average, but you cannot trust any single forecast: **variance**. The good forecaster uses the data enough to track the truth, but not so much that noise drives the forecast.

## Underfitting and overfitting: the symptoms

|                | Underfitting (high bias, low variance)                                        | Overfitting (low bias, high variance)                                                             |
|----------------|-------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| Cause          | Model too simple to capture the pattern; features do not capture the concept. | Model too complex; it learns the noise as well as the pattern; noisy or irrelevant features.      |
| Training error | <b>High</b> — cannot even fit the training data.                              | <b>Very low</b> ( $J \approx 0$ ).                                                                |
| Test error     | <b>High</b> , close to training error.                                        | <b>Very high</b> , far above training error — poor generalisation.                                |
| Picture        | $\theta_0 + \theta_1 x$ through a clearly curved cloud.                       | $\theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3 + \theta_4 x^4$ snaking through every point. |

The slide's "quality of fit" triptych: a line (underfit), a quadratic (correct fit), a quartic (overfit) on the same house-price points. The middle one is what we want, and there is no way to know it is the middle one *except by measuring test error*.

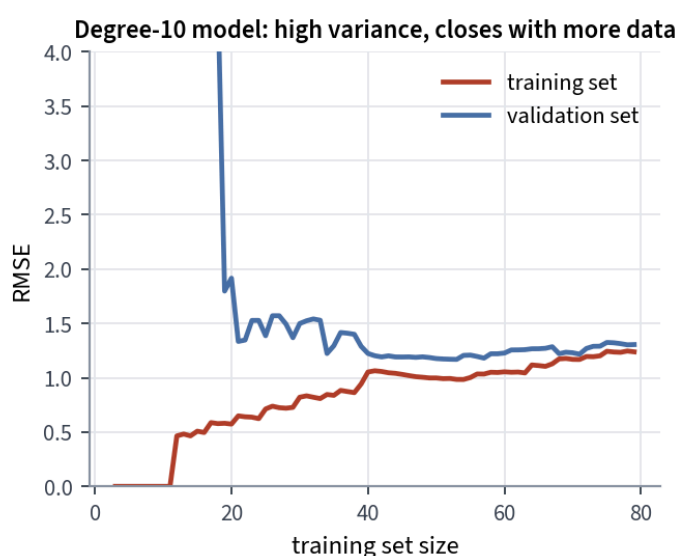
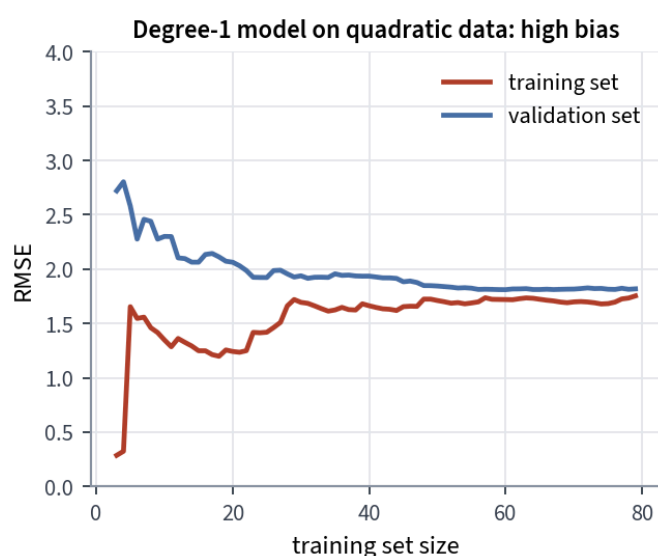
## The complexity curve



As complexity grows, training error keeps falling; test error falls, bottoms out, then rises. Left of the sweet spot is bias territory; right of it is variance territory.

This is the single most important picture in the module. Training error is not the objective; it always improves with complexity. The **test** (or validation) curve is what we minimise — and it is U-shaped. "How to experiment on model complexity" is exactly: try several complexities, plot both curves, pick the bottom of the U.

## Problem Type 4 — reading learning curves



Error against training-set size. Left: a too-simple model. Right: a too-flexible model.

**HOW TO READ THEM (THE SLIDE'S TWO PANELS)**

**Lower-order model:** both curves quickly flatten at a *high* error and stay close together. More data does not help — the model cannot represent the pattern. Diagnosis: **high bias, underfitting**. Cure: a more complex model or better features.

**Higher-order model:** training error starts near zero (a flexible model can fit a handful of points perfectly) and rises slowly; validation error starts very high and falls; a *gap* remains between them that *narrows* as the training set grows. Diagnosis: **high variance, overfitting**. Cure: more data (the gap closes), a simpler model, or regularisation. The slide's sentence: "size of the training dataset needs to be large to prevent [overfitting] when a higher-order model is used."

## Handling overfitting — three ways

**Way 1: more data**

The slide shows a degree-9 polynomial on  $N = 15$  points (wild) and on  $N = 100$  points (smooth, close to the true sine). With enough data, even a flexible model cannot bend to fit every noise point. Increasing the size of the dataset reduces overfitting.

**Way 2: a simpler model**

If the true relationship is quadratic, a linear model underfits with large bias and a degree-300 polynomial overfits with large variance; a quadratic is right. Choose complexity by the validation curve.

**Way 3: regularisation**

Keep the flexible model but **penalise large weights**, so it cannot wiggle. This is Chapter 10 — and it applies unchanged to logistic regression in Session 5–6.

One more picture from the slides: **error against epochs** of gradient descent. Training RMSE keeps falling; validation RMSE falls, reaches a minimum ("best model"), then slowly rises as the model starts fitting noise. Stopping at the minimum — **early stopping** — is a fourth, free way to limit overfitting.

**EXAM TIP**

Given a plot, always state three things: (1) what the curves do (training low/high, gap wide/narrow, does more data help?), (2) the diagnosis (high bias / high variance; underfitting / overfitting), (3) the remedy (more data, change complexity, regularise, early stopping). A diagnosis without the reason from the plot scores half.

**IN ONE BREATH**

Bias: too simple, high train and test error, more data will not help. Variance: too flexible, train error  $\approx 0$ , big train–test gap that closes with more data. Pick complexity at the bottom of the validation U; fight overfitting with more data, a simpler model, regularisation, or early stopping.

## CHAPTER 10

# Regularisation: Ridge, Lasso, Elastic Net

*Penalise big weights, derive the changed update rules, and finish the BMI problem with  $\lambda = 5$ .*

## STORY

Meera wants the flexibility of the degree-10 model without the madness. Arjun: "Charge it rent. Every unit of weight it uses costs something. Then it will only spend weight where the data really pays for it." That is regularisation.

## The idea

### BIG IDEA

Regularisation is a method for **automatically controlling the complexity** of the learned hypothesis. Add to the cost a penalty that grows with the size of the weights  $\theta_j$ . Minimising (fit + penalty) forces a trade: fit the data well, but not at the price of huge, wiggly weights. It works especially well when there are many features that each contribute a little.

### THINK OF IT LIKE

A budget for a school trip. Without a budget the organiser books everything (the model uses every feature at full strength). With a budget, each rupee must justify itself; frivolous items are cut first (small, noisy weights shrink toward zero) and only what clearly matters survives.

## Ridge regression (L2, Tikhonov)

### THE COST

$$J(\theta) = \underbrace{\frac{1}{2n} \sum_{i=1}^n \left( h_{\theta}(\mathbf{x}^{(i)}) - y^{(i)} \right)^2}_{\text{model fit to data}} + \underbrace{\frac{\lambda}{2} \sum_{j=1}^d \theta_j^2}_{\text{regularisation}}$$

$\lambda \geq 0$  is the **regularisation parameter** (a hyper-parameter). Note  $\sum_{j=1}^d \theta_j^2 = \|\boldsymbol{\theta}_{1:d}\|_2^2$ , the squared length of the weight vector — L2 regularisation pulls the coefficients toward **0**. **The intercept  $\theta_0$  is not penalised** (the sum starts at  $j = 1$ ): shifting the whole line up or down is not "complexity".

### MATHS, STEP BY STEP — THE RIDGE GRADIENT UPDATE

- 1 Differentiate the fit term as in Chapter 4:  $\frac{1}{n} \sum_i (h_{\theta}(\mathbf{x}^{(i)}) - y^{(i)}) x_j^{(i)}$ .
- 2 Differentiate the penalty:  $\frac{\partial}{\partial \theta_j} \frac{\lambda}{2} \sum_k \theta_k^2 = \lambda \theta_j$  for  $j \geq 1$ , and 0 for  $j = 0$ .

- 3 Plug into  $\theta_j \leftarrow \theta_j - \alpha \partial J / \partial \theta_j$ :

$$\theta_0 \leftarrow \theta_0 - \alpha \frac{1}{n} \sum_i (h_{\theta}(\mathbf{x}^{(i)}) - y^{(i)})$$

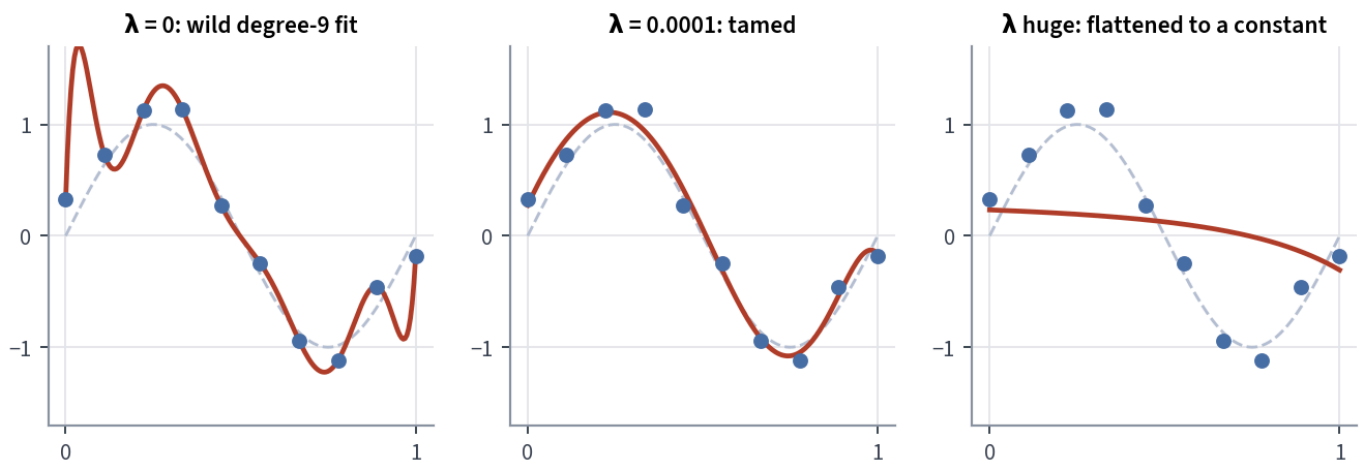
$$\theta_j \leftarrow \theta_j - \alpha \frac{1}{n} \sum_i (h_{\theta}(\mathbf{x}^{(i)}) - y^{(i)}) x_j^{(i)} - \alpha \lambda \theta_j \quad (j \geq 1)$$

- 4 Collect the two  $\theta_j$  terms — the form the slides highlight:

$$\theta_j \leftarrow \theta_j (1 - \alpha \lambda) - \alpha \frac{1}{n} \sum_i (h_{\theta}(\mathbf{x}^{(i)}) - y^{(i)}) x_j^{(i)}$$

Read it: **first shrink the weight by the factor  $(1 - \alpha \lambda)$ , then take the ordinary gradient step.** That is why L2 is also called *weight decay*. In matrix form the closed-form solution becomes  $\boldsymbol{\theta} = (\mathbf{X}^T \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^T \mathbf{y}$  with  $\mathbf{I}$  the identity with a 0 in the intercept position — which also cures the non-invertibility problem of Chapter 3.

## What $\lambda$ does



A degree-9 polynomial with three values of  $\lambda$ . Zero: the wild fit. Small: a smooth, sensible curve. Huge: all weights crushed toward zero, leaving only the intercept — a flat line.

The slide's thought experiment: set  $\lambda$  to something enormous like  $10^{10}$ . Then the penalty dominates and the only way to keep  $J$  small is  $\theta_1 \approx \theta_2 \approx \theta_3 \approx \theta_4 \approx 0$ , leaving  $h_{\theta}(x) \approx \theta_0$ : a horizontal line through the data — **underfitting**.  $\lambda = 0$  is ordinary least squares — possibly **overfitting**. In between is the sweet spot, chosen (like all hyper-parameters) on a validation set.

## Lasso regression (L1)

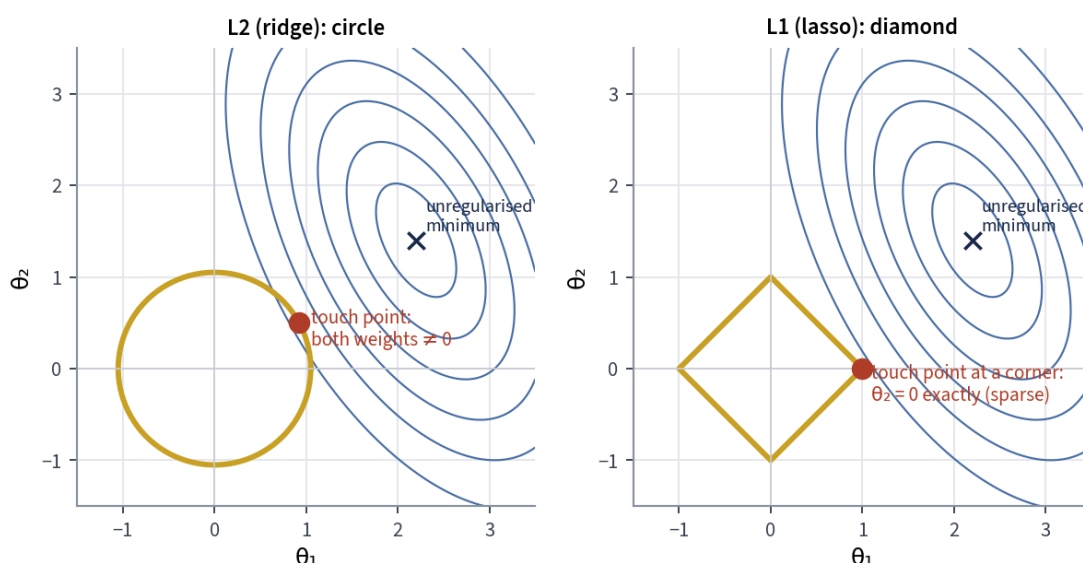
### THE COST AND THE UPDATE

$$J(\boldsymbol{\theta}) = \frac{1}{2n} \sum_{i=1}^n (h_{\theta}(\mathbf{x}^{(i)}) - y^{(i)})^2 + \lambda \sum_{j=1}^d |\theta_j|$$

The derivative of  $|\theta_j|$  is  $\text{sign}(\theta_j)$ :  $-1$  if  $\theta_j < 0$ ,  $+1$  if  $\theta_j > 0$  (and 0 at  $\theta_j = 0$  by convention). So

$$\theta_j \leftarrow \theta_j - \alpha \frac{1}{n} \sum_i (h_{\theta}(\mathbf{x}^{(i)}) - y^{(i)}) x_j^{(i)} - \alpha \lambda \text{sign}(\theta_j) \quad (j \geq 1)$$

Instead of shrinking proportionally, L1 subtracts a **fixed** amount  $\alpha\lambda$  in the direction of zero at every step. Small weights therefore reach *exactly* zero and stay there.



Why L1 makes weights exactly zero. Minimising (fit + penalty) is like finding where the fit's contours first touch the penalty's shape. A circle (L2) is usually touched on its curved side, where both weights are non-zero. A diamond (L1) is usually touched at a corner — on an axis — where one weight is exactly zero.

## Elastic net

$$J(\theta) = \frac{1}{2n} \sum_{i=1}^n (h_{\theta}(\mathbf{x}^{(i)}) - y^{(i)})^2 + r \frac{\lambda}{2} \sum_{j=1}^d \theta_j^2 + (1 - r) \lambda \sum_{j=1}^d |\theta_j|$$

A mix controlled by the **mix ratio**  $r$ :  $r = 0$  is lasso,  $r = 1$  is ridge (in scikit-learn the same knob is called `l1_ratio` and points the other way: `ElasticNet(alpha=0.1, l1_ratio=0.5)`). Preferred when features are highly correlated: lasso alone tends to pick one of a correlated group at random; the ridge part keeps the group together.

## Choosing among them

| L1 (lasso)                                                                                                                                                                                                     | L2 (ridge)                                                                                                            | Elastic net                                                           |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------|
| Can set some coefficients to <b>exactly 0</b> → a <b>sparse</b> model; effectively performs <b>feature selection</b> . Not differentiable at 0, so plain gradient methods are awkward (special solvers exist). | Shrinks all coefficients to <b>small</b> magnitudes but never exactly 0. Smooth, easy to optimise, has a closed form. | Both effects; the choice for <b>highly correlated</b> feature groups. |

## Worked example — the BMI problem with regularisation (slide)

### PROBLEM

Same data and start as Chapter 6 ( $\theta = (5, -0.03, -0.03)$ ,  $\alpha = 0.02$ ), now with regularisation constant  $\lambda = 5$ . Apply one iteration of ridge and of lasso, and interpret.

From Chapter 6 we already have the gradient pieces at the starting point:  $\frac{1}{3} \sum e = -0.08$ ,  $\frac{1}{3} \sum e x_1 = -3.8833$ ,  $\frac{1}{3} \sum e x_2 = -10.4667$ , so the plain gradient steps are  $-\alpha \cdot (\cdot)$ :  $+0.0016$ ,  $+0.07767$ ,  $+0.20933$ .

**Ridge** ( $\theta_j \leftarrow \theta_j(1 - \alpha\lambda) - \alpha \text{grad}_j$ ;  $\alpha\lambda = 0.02 \times 5 = 0.1$ , so the shrink factor is 0.9):

- 1  $\theta'_0 = 5 + 0.0016 = 5.0016$  (intercept not shrunk)
- 2  $\theta'_1 = -0.03(0.9) + 0.07767 = -0.027 + 0.07767 = \mathbf{0.0507}$
- 3  $\theta'_2 = -0.03(0.9) + 0.20933 = -0.027 + 0.20933 = \mathbf{0.1823}$

**Lasso** ( $\theta_j \leftarrow \theta_j - \alpha \text{grad}_j - \alpha\lambda \text{sign}(\theta_j)$ ; both weights start negative, so  $\text{sign} = -1$  and the L1 term adds 0.1):

- 1  $\theta'_0 = 5.0016$
- 2  $\theta'_1 = -0.03 + 0.07767 - 0.1(-1) = 0.04767 + 0.1 = \mathbf{0.1477}$
- 3  $\theta'_2 = -0.03 + 0.20933 + 0.1 = \mathbf{0.2793}$

| Model                      | $w_0$  | $w_1$  | $w_2$  |
|----------------------------|--------|--------|--------|
| Linear (no regularisation) | 5.0016 | 0.0477 | 0.1793 |
| Ridge                      | 5.0016 | 0.0507 | 0.1823 |
| Lasso                      | 5.0016 | 0.1477 | 0.2793 |

**Interpretation (slide).** Without regularisation the weights jump from  $-0.03$  to  $+0.05$  and  $+0.18$  — large moves. Ridge first multiplies the old weight by 0.9 (a small shrink of  $-0.03 \rightarrow -0.027$ ), so its result differs from linear by only 0.003. Lasso adds a fixed  $\alpha\lambda = 0.1$  toward zero from wherever the weight is — which here, because the old weights were slightly negative, pushes them *through* zero to  $+0.15$  and  $+0.28$ .

### CAREFUL

Two things to say in an exam about this example. (1) The L1 step of size  $\alpha\lambda = 0.1$  is bigger than the weights themselves (0.03), so the "push toward zero" overshoots; real lasso solvers use *soft-thresholding* (stop at zero instead of crossing it). (2) With unscaled features the second iteration diverges for all three models (Chapter 6) — regularisation does not fix a bad learning rate.

**EXAM TIP**

Write the modified update rule before the numbers: ridge multiplies the old weight by  $(1 - \alpha\lambda)$ ; lasso subtracts  $\alpha\lambda \text{sign}(\theta_j)$ ; the intercept is never regularised. Then reuse the gradient sums from the unregularised step — they are identical at the same  $\theta$ .

**IN ONE BREATH**

Ridge: penalty  $\frac{\lambda}{2} \sum \theta_j^2$ , update  $\theta_j \leftarrow \theta_j(1 - \alpha\lambda) - \alpha \text{grad}_j$  — shrinks. Lasso: penalty  $\lambda \sum |\theta_j|$ , update subtracts  $\alpha\lambda \text{sign}(\theta_j)$  — zeroes out. Elastic net mixes them with ratio  $r$ . Never penalise  $\theta_0$ ; pick  $\lambda$  on validation data.

## CHAPTER 11

# Formula Sheet, Self-Check, and What Comes Next

Every formula from Sessions 3–4 on one page, twelve questions with answers, and the bridge to classification.

## The formula sheet

### Model and cost

$$h_{\theta}(\mathbf{x}) = \theta_0 + \theta_1 x_1 + \cdots + \theta_d x_d = \boldsymbol{\theta}^T \mathbf{x} \quad (x_0 = 1); \quad J(\boldsymbol{\theta}) = \frac{1}{2n} \sum_{i=1}^n (h_{\theta}(\mathbf{x}^{(i)}) - y^{(i)})^2 = \frac{1}{2n} (\mathbf{X}\boldsymbol{\theta} - \mathbf{y})^T (\mathbf{X}\boldsymbol{\theta} - \mathbf{y})$$

### Closed form

One input:  $\theta_1 = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sum (x_i - \bar{x})^2}$ ,  $\theta_0 = \bar{y} - \theta_1 \bar{x}$ . General:  $\boldsymbol{\theta} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$ .  $2 \times 2$  inverse:

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix}^{-1} = \frac{1}{ad-bc} \begin{bmatrix} d & -b \\ -c & a \end{bmatrix}.$$

### Gradient descent

$$\frac{\partial J}{\partial \theta_j} = \frac{1}{n} \sum_i (h_{\theta}(\mathbf{x}^{(i)}) - y^{(i)}) x_j^{(i)}; \quad \text{batch: } \theta_j \leftarrow \theta_j - \alpha \frac{1}{n} \sum_i (\cdot) x_j^{(i)}; \quad \text{SGD: } \theta_j \leftarrow \theta_j - \alpha (h_{\theta}(\mathbf{x}^{(i)}) - y^{(i)}) x_j^{(i)}; \quad \text{matrix: } \boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \frac{\alpha}{n} \mathbf{X}^T (\mathbf{X}\boldsymbol{\theta} - \mathbf{y}).$$

### Evaluation

$$\text{MSE} = \frac{1}{N} \sum (y_i - \hat{y}_i)^2, \quad \text{RMSE} = \sqrt{\text{MSE}}, \quad \text{MAE} = \frac{1}{N} \sum |y_i - \hat{y}_i|, \quad R^2 = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}.$$

### Basis functions and regularisation

$$y = \sum_j w_j \phi_j(\mathbf{x}) = \mathbf{w}^T \boldsymbol{\phi}(\mathbf{x}); \quad \text{ridge: } J + \frac{\lambda}{2} \sum_{j \geq 1} \theta_j^2, \quad \theta_j \leftarrow \theta_j (1 - \alpha \lambda) - \alpha \text{grad}_j; \quad \text{lasso: } J + \lambda \sum_{j \geq 1} |\theta_j|, \quad \theta_j \leftarrow \theta_j - \alpha \text{grad}_j - \alpha \lambda \text{sign}(\theta_j); \quad \text{elastic net: } r \frac{\lambda}{2} \sum \theta_j^2 + (1-r) \lambda \sum |\theta_j|.$$

## Twelve questions (cover the right column)

| #  | Question                                                                               | Answer                                                                                                                             |
|----|----------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|
| 1  | Why is there a $\frac{1}{2}$ in $J$ ? Does it change the answer?                       | It cancels the 2 from differentiating a square. No — the minimiser is unchanged.                                                   |
| 2  | Why does least squares have no local minima?                                           | $J$ is convex (a bowl); a convex function has a single global minimum.                                                             |
| 3  | State the normal equation and its main cost.                                           | $\theta = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$ ; inverting a $(d+1) \times (d+1)$ matrix, roughly $O(d^3)$ .    |
| 4  | When is $\mathbf{X}^T \mathbf{X}$ singular and what do you do?                         | Redundant (linearly dependent) features or $d > n$ ; use the pseudo-inverse, drop redundant features, or regularise.               |
| 5  | Why does $\theta \leftarrow \theta - \alpha \partial J / \partial \theta$ go downhill? | Positive slope $\Rightarrow$ subtract $\Rightarrow$ move left; negative slope $\Rightarrow$ move right; both reduce $J$ .          |
| 6  | What does "simultaneous update" mean?                                                  | Compute all gradients with the old $\theta$ , then update all $\theta_j$ together.                                                 |
| 7  | $J$ rises every iteration. Diagnosis and fix?                                          | $\alpha$ too large (overshooting); reduce $\alpha$ (and check feature scaling).                                                    |
| 8  | Why scale features before gradient descent?                                            | Unequal scales stretch the contours into a valley; descent zig-zags. Scaling makes contours circular and convergence fast.         |
| 9  | Batch vs stochastic vs mini-batch — updates per epoch?                                 | 1, $n$ , $n/\text{batch\_size}$ ; speed fast $\rightarrow$ slow: SGD, mini-batch, batch; stability the reverse.                    |
| 10 | Why is polynomial regression a linear model?                                           | Prediction is linear in the weights $\mathbf{w}$ ; the non-linearity lives in fixed basis functions $\phi_j(x) = x^j$ .            |
| 11 | Training error tiny, test error huge. Name it, cure it.                                | Overfitting / high variance; more data, simpler model, regularisation, early stopping.                                             |
| 12 | Which regulariser gives exact zeros, and why?                                          | L1 (lasso): the fixed step $\alpha \lambda \text{sign}(\theta_j)$ drives small weights to exactly 0 (diamond corners on the axes). |

### SPARK

Everything in this book was one sentence, repeated: **write a cost, then make it small**. The cost was squared error; making it small was either algebra (set the derivative to zero) or walking (gradient descent). Change the cost to a penalty-plus-fit and you get ridge and lasso. In the next session the *same* two moves — a different cost, the same gradient descent — will give you a classifier. You already know 80% of logistic regression.

**STORY — TO BE CONTINUED**

Meera's regularised price model ships. Test RMSE: ₹38,000,  $R^2 = 0.81$ . The board is happy. Then Finance walks in with a different question: not "how much?" but "will this borrower default — yes or no?" A line will not do. That is Book 4.